

Contents

M3 - Desarrollo de APIs REST con Spring Boot	5
Punto 3.1 - GET: consultar recursos	5
Objetivos de aprendizaje	5
Bloque 1 - Qué significa consultar un recurso	5
T1.1 - El método GET y su semántica	5
T1.2 - GET de colección vs GET individual	6
T1.3 - Filtrado, ordenación y paginación	6
Actualización 2026 y vínculo con el proyecto	6
Pregunta	6
Respuesta razonada	6
Bloque 2 - Parámetros de ruta y query	7
T2.1 - @PathVariable	7
T2.2 - @RequestParam	7
T2.3 - Combinación de ambos	7
Actualización 2026 y vínculo con el proyecto	8
Pregunta	8
Respuesta razonada	8
Bloque 3 - Filtrado con streams	8
T3.1 - Filtrado simple	8
T3.2 - Filtrado con múltiples criterios	9
T3.3 - Ordenación con streams	9
Actualización 2026 y vínculo con el proyecto	9
Pregunta	9
Respuesta razonada	10
Bloque 4 - Paginación manual	10
T4.1 - Qué es paginar	10
T4.2 - Implementación manual con skip y limit	10
T4.3 - Metadatos de paginación	10
Actualización 2026 y vínculo con el proyecto	11
Pregunta	11
Respuesta razonada	11
Bloque 5 - Códigos de estado y buenas prácticas	11
T5.1 - Códigos de estado en GET	11
T5.2 - Buenas prácticas en GET	11
T5.3 - Tests para GET	11
Actualización 2026 y vínculo con el proyecto	12
Pregunta	12
Respuesta razonada	12
Resumen del punto	12
Puente a la práctica	12
Punto 3.2 - POST: crear recursos	13
Objetivos de aprendizaje	13
Bloque 1 - Qué significa crear un recurso	13
T1.1 - El método POST y su semántica	13
T1.2 - La URL y el cuerpo	13
T1.3 - Qué devuelve el servidor	13
Actualización 2026 y vínculo con el proyecto	14
Pregunta	14
Respuesta razonada	14
Bloque 2 - @RequestBody y deserialización	14
T2.1 - Qué hace @RequestBody	14
T2.2 - Cómo Jackson deserializa	15
T2.3 - Validación manual en el servicio	15

Actualización 2026 y vínculo con el proyecto	15
Pregunta	15
Respuesta razonada	15
Bloque 3 - Códigos de estado y cabecera Location	16
T3.1 - 201 Created	16
T3.2 - La cabecera Location	16
T3.3 - Conflictos y errores	16
Actualización 2026 y vínculo con el proyecto	16
Pregunta	16
Respuesta razonada	17
Bloque 4 - Validación de datos	17
T4.1 - Validación de formato vs validación de negocio	17
T4.2 - Validación de negocio en el servicio	17
T4.3 - Manejo de la excepción en el controlador	17
Actualización 2026 y vínculo con el proyecto	18
Pregunta	18
Respuesta razonada	18
Bloque 5 - Buenas prácticas y tests	18
T5.1 - Buenas prácticas en POST	18
T5.2 - Tests para POST	18
T5.3 - La cabecera Location en los tests	18
Actualización 2026 y vínculo con el proyecto	19
Pregunta	19
Respuesta razonada	19
Resumen del punto	19
Puente a la práctica	19
Punto 3.3 - PUT y PATCH: actualizar recursos	19
Objetivos de aprendizaje	19
Bloque 1 - Qué significa actualizar un recurso	20
T1.1 - Los dos sabores de la actualización	20
T1.2 - PUT: reemplazo completo	20
T1.3 - PATCH: actualización parcial	20
Actualización 2026 y vínculo con el proyecto	21
Pregunta	21
Respuesta razonada	21
Bloque 2 - PUT en profundidad	21
T2.1 - Semántica de PUT	21
T2.2 - Implementación de PUT en el servicio	21
T2.3 - Implementación de PUT en el controlador	22
Actualización 2026 y vínculo con el proyecto	22
Pregunta	22
Respuesta razonada	22
Bloque 3 - PATCH en profundidad	22
T3.1 - Semántica de PATCH	22
T3.2 - Implementación de PATCH en el servicio	23
T3.3 - Implementación de PATCH en el controlador	23
Actualización 2026 y vínculo con el proyecto	24
Pregunta	24
Respuesta razonada	24
Bloque 4 - Códigos de estado y errores	24
T4.1 - Códigos de estado en PUT y PATCH	24
T4.2 - Validación en actualizaciones	24
T4.3 - Manejo de errores en actualizaciones	25
Actualización 2026 y vínculo con el proyecto	25
Pregunta	25
Respuesta razonada	25

Bloque 5 - Buenas prácticas y tests	25
T5.1 - Buenas prácticas en PUT y PATCH	25
T5.2 - Tests para PUT y PATCH	26
T5.3 - Verificar la inmutabilidad de campos no enviados en PATCH	26
Actualización 2026 y vínculo con el proyecto	26
Pregunta	26
Respuesta razonada	27
Resumen del punto	27
Puente a la práctica	27
Punto 3.4 - DELETE: eliminar recursos	27
Objetivos de aprendizaje	27
Bloque 1 - Qué significa eliminar un recurso	27
T1.1 - El método DELETE y su semántica	27
T1.2 - La URL de un DELETE	27
T1.3 - Qué devuelve el servidor	28
Actualización 2026 y vínculo con el proyecto	28
Pregunta	28
Respuesta razonada	28
Bloque 2 - Hard delete vs soft delete	28
T2.1 - Qué es hard delete	28
T2.2 - Qué es soft delete	29
T2.3 - Cuando usar cada uno	29
Actualización 2026 y vínculo con el proyecto	29
Pregunta	30
Respuesta razonada	30
Bloque 3 - Implementación de DELETE	30
T3.1 - Implementación en el repositorio	30
T3.2 - Implementación en el servicio	30
T3.3 - Implementación en el controlador	30
Actualización 2026 y vínculo con el proyecto	31
Pregunta	31
Respuesta razonada	31
Bloque 4 - Dependencias entre recursos	31
T4.1 - El problema de las dependencias	31
T4.2 - Implementación de restricción	31
T4.3 - Implementación de cascada	32
Actualización 2026 y vínculo con el proyecto	32
Pregunta	32
Respuesta razonada	32
Bloque 5 - Buenas prácticas y tests	32
T5.1 - Buenas prácticas en DELETE	32
T5.2 - Tests para DELETE	32
T5.3 - La idempotencia en los tests	33
Actualización 2026 y vínculo con el proyecto	33
Pregunta	33
Respuesta razonada	33
Resumen del punto	33
Puente a la práctica	34
Punto 3.5 - DTOs avanzados y validaciones	34
Objetivos de aprendizaje	34
Bloque 1 - DTOs de entrada y de salida	34
T1.1 - Por qué separar DTOs de entrada y de salida	34
T1.2 - DTO de entrada	34
T1.3 - DTO de salida	35
Actualización 2026 y vínculo con el proyecto	35

Pregunta	35
Respuesta razonada	36
Bloque 2 - DTOs anidados	36
T2.1 - Qué es un DTO anidado	36
T2.2 - Cuándo usar DTOs anidados	37
T2.3 - DTOs anidados con listas	37
Actualización 2026 y vínculo con el proyecto	38
Pregunta	38
Respuesta razonada	38
Bloque 3 - Bean Validation	38
T3.1 - Qué es Bean Validation	38
T3.2 - Anotaciones de validación	38
T3.3 - Activar la validación con @Valid	39
Actualización 2026 y vínculo con el proyecto	39
Pregunta	39
Respuesta razonada	39
Bloque 4 - Manejo de errores de validación	39
T4.1 - Qué es MethodArgumentNotValidException	39
T4.2 - Extraer los errores	40
T4.3 - Construir la respuesta de error	40
Actualización 2026 y vínculo con el proyecto	40
Pregunta	41
Respuesta razonada	41
Bloque 5 - Buenas prácticas y tests	41
T5.1 - Buenas prácticas en DTOs y validaciones	41
T5.2 - Tests para validaciones	41
T5.3 - Verificar los mensajes de error en los tests	41
Actualización 2026 y vínculo con el proyecto	42
Pregunta	42
Respuesta razonada	42
Resumen del punto	42
Puente a la práctica	42
Punto 3.6 - Parámetros, cabeceras y Postman	42
Objetivos de aprendizaje	42
Bloque 1 - @RequestParam en profundidad	43
T1.1 - Recordatorio y variantes	43
T1.2 - Listas y mapas como parámetros	43
T1.3 - @RequestParam con tipos no String	43
Actualización 2026 y vínculo con el proyecto	44
Pregunta	44
Respuesta razonada	44
Bloque 2 - @RequestHeader	44
T2.1 - Qué son las cabeceras HTTP	44
T2.2 - @RequestHeader	44
T2.3 - Casos de uso típicos	45
Actualización 2026 y vínculo con el proyecto	45
Pregunta	45
Respuesta razonada	45
Bloque 3 - @CookieValue y @MatrixVariable	46
T3.1 - Qué son las cookies	46
T3.2 - @CookieValue	46
T3.3 - @MatrixVariable	46
Actualización 2026 y vínculo con el proyecto	47
Pregunta	47
Respuesta razonada	47
Bloque 4 - Postman	47

T4.1 - Qué es Postman y por qué usarlo	47
T4.2 - Colecciones y entornos	47
T4.3 - Tests automáticos en Postman	47
Actualización 2026 y vínculo con el proyecto	48
Pregunta	48
Respuesta razonada	48
Bloque 5 - Buenas prácticas con parámetros y cabeceras	48
T5.1 - Buenas prácticas con @RequestParam	48
T5.2 - Buenas prácticas con @RequestHeader	48
T5.3 - Buenas prácticas con Postman	49
Actualización 2026 y vínculo con el proyecto	49
Pregunta	49
Respuesta razonada	49
Resumen del punto	49
Puente a la práctica	49

M3 - Desarrollo de APIs REST con Spring Boot

Este módulo continúa desde el snapshot final de M2. El proyecto ya tiene capas por funcionalidad, tests, perfiles, OpenAPI y controles de calidad; M3 utiliza esa base para convertir el CRUD existente en un contrato REST más completo y verificable.

La edición conserva toda la progresión del PDF fuente - GET, POST, PUT/PATCH, DELETE, DTOs/validación y parámetros/cabeceras/Postman - y añade notas de actualización para Java 17, Spring Boot 3.5.16, Maven Wrapper 3.9.16 y las decisiones que realmente aparecen en el proyecto acumulativo.

Punto 3.1 - GET: consultar recursos

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de: 1. Explicar qué significa consultar un recurso y por qué GET es el método adecuado. 2. Diferenciar GET de colección y GET individual, y sus códigos de estado. 3. Implementar filtrado, ordenación y paginación en un endpoint de colección. 4. Usar correctamente @PathVariable y @RequestParam. 5. Construir respuestas con ResponseEntity para GET. 6. Escribir tests para endpoints GET. 7. Diagnosticar y resolver los errores más comunes al implementar GET.

Bloque 1 - Qué significa consultar un recurso

T1.1 - El método GET y su semántica

GET es el método HTTP que significa “quiero leer esto, no modificarlo”. Es el método que usa el navegador cuando escribes una URL en la barra de direcciones. Es el método que usan los clientes cuando quieren consultar datos sin cambiar nada. GET tiene tres propiedades que lo definen:

- Es seguro. No modifica el estado del servidor. Puedes hacer un GET mil veces y el servidor no cambia.
- Es idempotente. Ejecutarlo N veces produce el mismo resultado que ejecutarlo una vez. Si haces cinco GET al mismo recurso, recibes lo mismo cinco veces.
- Es cacheable. Los proxies y los navegadores pueden guardar la respuesta y reutilizarla. Eso mejora el rendimiento. Estas propiedades no son un capricho: son un contrato. Si tu endpoint GET modifica algo, estás rompiendo el contrato. Los navegadores pueden pre-cargar URLs GET, los crawlers pueden rastrearlas, los proxies pueden cachearlas. Si un GET tuviera efectos secundarios, esas acciones podrían causar estragos. Por eso, en una API REST, GET solo se

usa para consultar. Nunca para crear, actualizar o eliminar. Para esas operaciones hay otros métodos.

T1.2 - GET de colección vs GET individual

Hay dos tipos de GET en una API REST: GET de colección. Devuelve una lista de recursos. La URL apunta a la colección: `/api/v1/alumnos`. El cliente recibe un array JSON con todos los alumnos (o un subconjunto, si hay paginación). GET individual. Devuelve un recurso concreto. La URL apunta al recurso: `/api/v1/alumnos/{id}`. El cliente recibe un objeto JSON con el alumno. Los dos tienen códigos de estado distintos:

- GET de colección: siempre devuelve 200 OK, incluso si la colección está vacía (en ese caso, devuelve un array vacío []). No devuelve 404 porque la colección existe aunque no tenga elementos.
- GET individual: devuelve 200 OK si el recurso existe, y 404 Not Found si no existe. No devuelve 200 con null: eso confundiría al cliente. Esta distinción es importante. Un error común es devolver 404 en un GET de colección cuando no hay elementos. Eso es incorrecto: la colección existe, solo está vacía. El cliente debe recibir [] con un 200.

T1.3 - Filtrado, ordenación y paginación

Un GET de colección no siempre devuelve todos los recursos. Muchas veces el cliente quiere un subconjunto. Para eso están el filtrado, la ordenación y la paginación. Filtrado. El cliente indica criterios para seleccionar un subconjunto. Por ejemplo: `/api/v1/alumnos?curso=5º` Primaria devuelve solo los alumnos de 5º. Los filtros van en query parameters, no en la ruta. Ordenación. El cliente indica en qué orden quiere los resultados. Por ejemplo: `/api/v1/alumnos?sort=nombre` devuelve los alumnos ordenados por nombre. El orden puede ser ascendente (asc) o descendente (desc). Paginación. El cliente indica qué página quiere y cuántos elementos por página. Por ejemplo: `/api/v1/alumnos?page=0&size=20` devuelve los primeros 20 alumnos. La paginación evita enviar colecciones gigantes que consumen ancho de banda y tardan en serializar. Los tres se combinan: `/api/v1/alumnos?curso=5º Primaria&sort=nombre&page=0&size=10`. En este punto implementaremos filtrado y ordenación de forma manual (con streams de Java). En el Módulo 4, cuando usemos Spring Data JPA, veremos cómo hacerlo de forma más eficiente con Pageable y Sort.

Pregunta: ¿Por qué el filtrado va en query parameters y no en la ruta? ¿Qué problema tendría `/api/v1/alumnos/curso/5º`?

Actualización 2026 y vínculo con el proyecto

GET representa lectura sin efectos laterales. Una colección y un elemento individual son recursos distintos aunque compartan raíz. El contrato debe separar recurso existente, ausencia, colección vacía y petición inválida.

En el snapshot acumulativo de M3 esta idea no queda como teoría aislada: se refleja en el controlador, el servicio, los DTO y los tests que corresponden a este punto. La comprobación debe hacerse sobre el comportamiento HTTP observable y no sólo leyendo anotaciones.

Pregunta

¿Qué decisión de contrato de **qué significa consultar un recurso** debería poder demostrar un test o una petición reproducible?

Respuesta razonada

GET representa lectura sin efectos laterales. Una colección y un elemento individual son recursos distintos aunque compartan raíz. El contrato debe separar recurso existente, ausencia, colección vacía y petición inválida. La evidencia correcta combina una entrada concreta con el status, las cabeceras y/o el cuerpo esperados, y deja claro qué capa aplica la regla.

Bloque 2 - Parámetros de ruta y query

T2.1 - @PathVariable

@PathVariable extrae un valor de la ruta de la URL y lo inyecta como parámetro del método. Se usa para capturar identificadores de recursos. Por ejemplo, en un método mapeado con @GetMapping("/{id}"):

```
@GetMapping("/{id}")
public ResponseEntity<AlumnoDTO> consultar(@PathVariable String id) {
    // ...
}
```

El {id} en la ruta se captura con @PathVariable String id. Si el cliente visita /api/v1/alumnos/12345, el parámetro id valdrá "12345". El nombre de la variable en la ruta y el nombre del parámetro deben coincidir. Si no coinciden, se puede especificar explícitamente:

```
@GetMapping("/{id}")
public ResponseEntity<AlumnoDTO> consultar(@PathVariable("id") String identificador) {
    // ...
}
```

Los @PathVariable son obligatorios por defecto. Si la ruta no incluye el valor, Spring MVC no mapea la petición a ese método. No se puede hacer un GET individual sin ID.

T2.2 - @RequestParam

@RequestParam extrae un valor de la query string de la URL y lo inyecta como parámetro del método. Se usa para filtros, ordenación y paginación. Por ejemplo:

```
@GetMapping
public List<AlumnoDTO> listar(@RequestParam(required = false) String curso) {
    // ...
}
```

Si el cliente visita /api/v1/alumnos?curso=5º Primaria, el parámetro curso valdrá "5º Primaria". Si visita /api/v1/alumnos sin query, curso será null porque hemos puesto required = false. Por defecto, @RequestParam es obligatorio: si el cliente no envía el parámetro, Spring MVC devuelve un 400 Bad Request. Para hacerlo opcional, se usa required = false. Para darle un valor por defecto, se usa defaultValue:

```
@GetMapping
public List<AlumnoDTO> listar(
    @RequestParam(required = false) String curso,
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "20") int size) {
    // ...
}
```

Aquí curso es opcional (será null si no se envía), mientras que page y size tienen valores por defecto (0 y 20). Si el cliente no los envía, se usan esos valores.

T2.3 - Combinación de ambos

Un endpoint puede combinar @PathVariable y @RequestParam. Por ejemplo, un GET de sub-recurso con filtros:

```
@GetMapping("/{id}/documentos")
public List<DocumentoDTO> listarDocumentos(
    @PathVariable String id,
    @RequestParam(required = false) String tipo) {
```

```
// ...
}
```

La URL `/api/v1/alumnos/12345/documentos?tipo=PDF` captura id de la ruta y tipo de la query. Cada uno tiene su función: el path identifica el recurso padre; la query filtra los sub-recursos. La regla es clara:

- Path parameter: identifica un recurso. Va en la ruta. Es obligatorio.
- Query parameter: filtra, ordena o pagina. Va en la query. Es opcional. **Pregunta: ¿Por qué el ID de un recurso va en la ruta y no en la query? ¿Qué diferencia hay entre `/alumnos/1` y `/alumnos?id=1`?**

Actualización 2026 y vínculo con el proyecto

@PathVariable identifica una instancia; @RequestParam modifica la vista de una colección. Los filtros deben ser opcionales, combinables y coherentes. Las fechas ISO se convierten con @DateFormat.

En el snapshot acumulativo de M3 esta idea no queda como teoría aislada: se refleja en el controlador, el servicio, los DTO y los tests que corresponden a este punto. La comprobación debe hacerse sobre el comportamiento HTTP observable y no sólo leyendo anotaciones.

Pregunta

¿Qué decisión de contrato de **parámetros de ruta y query** debería poder demostrar un test o una petición reproducible?

Respuesta razonada

@PathVariable identifica una instancia; @RequestParam modifica la vista de una colección. Los filtros deben ser opcionales, combinables y coherentes. Las fechas ISO se convierten con @DateFormat. La evidencia correcta combina una entrada concreta con el status, las cabeceras y/o el cuerpo esperados, y deja claro qué capa aplica la regla.

Bloque 3 - Filtrado con streams

T3.1 - Filtrado simple

El filtrado se implementa en el servicio, no en el controlador. El controlador recibe el query parameter y lo pasa al servicio. El servicio decide cómo filtrar. Con un repositorio en memoria (como el nuestro), el filtrado se hace con streams de Java. Un stream es una secuencia de elementos sobre la que se pueden aplicar operaciones (filtrar, ordenar, transformar). Para filtrar por curso:

```
public List<AlumnoDTO> listarPorCurso(String curso) {
    return repositorio.listarTodos().stream()
        .filter(a -> a.getCurso().equalsIgnoreCase(curso))
        .toList();
}
```

- `.stream()` convierte la lista en un stream.
- `.filter(a -> ...)` mantiene sólo los elementos que cumplen la condición.
- `.toList()` recoge el resultado en una lista.

La lambda `a -> a.getCurso().equalsIgnoreCase(curso)` es un predicado: recibe un alumno y devuelve true si su curso coincide

(ignorando mayúsculas). `equalsIgnoreCase` compara strings sin distinguir mayúsculas y minúsculas.

T3.2 - Filtrado con múltiples criterios

Cuando hay varios filtros, se pueden combinar en el servicio. Por ejemplo, filtrar por curso y por DNI:

```
public List<AlumnoDTO> listarConFiltros(String curso, String dni) {
    return repositorio.listarTodos().stream()
        .filter(a -> curso == null || a.getCurso().equalsIgnoreCase(curso))
        .filter(a -> dni == null || a.getDni().equals(dni))
        .toList();
}
```

Cada filter se aplica en cadena. Si un filtro es null, se ignora (la condición es true). Así, si el cliente no envía curso, no se filtra por curso. Esta técnica se llama filtrado condicional: los filtros se aplican solo si el parámetro está presente. Es muy común en APIs REST.

T3.3 - Ordenación con streams

La ordenación también se hace con streams, usando .sorted():

```
public List<AlumnoDTO> listarOrdenado(String sort) {
    return repositorio.listarTodos().stream()
        .sorted((a, b) -> {
            if ("nombre".equals(sort)) {
                return a.getNombre().compareToIgnoreCase(b.getNombre());
            }
            if ("apellidos".equals(sort)) {
                return a.getApellidos().compareToIgnoreCase(b.getApellidos());
            }
            return 0;
        })
        .toList();
}
```

.sorted((a, b) -> ...) ordena el stream usando un comparador. El comparador recibe dos elementos y devuelve un entero

negativo, cero o positivo según su orden. La lambda comprueba qué campo se quiere ordenar y aplica el comparador correspondiente. compareToIgnoreCase compara strings sin distinguir mayúsculas. Si no se reconoce el campo, se devuelve 0, lo que deja el orden original.

Pregunta: ¿Qué diferencia hay entre compareTo y compareToIgnoreCase? ¿Cuándo usarías cada uno?

Actualización 2026 y vínculo con el proyecto

filter compone predicados sin duplicar rutas. La ordenación debe aceptar un vocabulario explícito y manejar nulos. El mismo conjunto filtrado debe alimentar contenido y conteo para evitar metadatos incoherentes.

En el snapshot acumulativo de M3 esta idea no queda como teoría aislada: se refleja en el controlador, el servicio, los DTO y los tests que corresponden a este punto. La comprobación debe hacerse sobre el comportamiento HTTP observable y no sólo leyendo anotaciones.

Pregunta

¿Qué decisión de contrato de **filtrado con streams** debería poder demostrar un test o una petición reproducible?

Respuesta razonada

filter compone predicados sin duplicar rutas. La ordenación debe aceptar un vocabulario explícito y manejar nulos. El mismo conjunto filtrado debe alimentar contenido y conteo para evitar metadatos incoherentes. La evidencia correcta combina una entrada concreta con el status, las cabeceras y/o el cuerpo esperados, y deja claro qué capa aplica la regla.

Bloque 4 - Paginación manual

T4.1 - Qué es paginar

Paginar es dividir una colección grande en trozos más pequeños, llamados páginas. El cliente pide una página concreta y el servidor devuelve solo ese trozo. La paginación tiene dos parámetros:

- page: el número de página, empezando en 0.
- size: el número de elementos por página. Por ejemplo, si hay 47 alumnos y el cliente pide page=0&size=20, recibe los alumnos del 0 al 19. Si pide page=1&size=20, recibe los del 20 al 39. Si pide page=2&size=20, recibe los del 40 al 46 (solo 7). La paginación evita enviar colecciones gigantes que consumen ancho de banda y tardan en serializar. También permite al cliente construir controles de navegación (siguiente, anterior, ir a página).

T4.2 - Implementación manual con skip y limit

Con streams de Java, la paginación se implementa con .skip() y .limit():

```
public List<AlumnoDTO> listarPaginado(int page, int size) {  
    return repositorio.listarTodos().stream()  
        .skip((long) page * size)  
        .limit(size)  
        .toList();  
}
```

.skip(n) descarta los primeros n elementos. El número de elementos a descartar es page * size (si estamos en la página 2 con

tamaño 20, descartamos los primeros 40). .limit(size) limita el resultado a size elementos. El cast (long) es necesario porque page * size puede desbordar un int si los valores son grandes. skip espera un long. Esta implementación es manual y sencilla. En el Módulo 4, con Spring Data JPA, veremos cómo hacerlo de forma más eficiente con Pageable, que además devuelve metadatos (total de elementos, total de páginas).

T4.3 - Metadatos de paginación

Una respuesta paginada no solo incluye los elementos de la página actual, sino también metadatos que permiten al cliente navegar:

```
{  
  "content": [  
    {"id": "1", "nombre": "Ana"},  
    {"id": "2", "nombre": "Luis"}  
  ],  
  "page": 0,  
  "size": 20,  
  "totalElements": 47,  
  "totalPages": 3  
}
```

- content: los elementos de la página.
- page: el número de página actual.
- size: el tamaño de la página.

- `totalElements`: el número total de elementos.
 - `totalPages`: el número total de páginas. Con estos metadatos, el cliente sabe cuántos elementos hay en total, cuántas páginas hay y en qué página está. Puede construir controles de navegación sin hacer más peticiones. En este punto implementaremos una versión simple de estos metadatos. En el Módulo 4, con `Pageable`, Spring Data los genera automáticamente.
- Pregunta: ¿Qué ventaja tiene incluir metadatos de paginación en la respuesta?**

Actualización 2026 y vínculo con el proyecto

`page` es índice base cero y `size` limita el lote. `skip(page*size)` y `limit(size)` sirven en memoria; `totalElements` y `totalPages` forman parte del contrato. Se validan límites antes de operar.

En el snapshot acumulativo de M3 esta idea no queda como teoría aislada: se refleja en el controlador, el servicio, los DTO y los tests que corresponden a este punto. La comprobación debe hacerse sobre el comportamiento HTTP observable y no sólo leyendo anotaciones.

Pregunta

¿Qué decisión de contrato de **paginación manual** debería poder demostrar un test o una petición reproducible?

Respuesta razonada

`page` es índice base cero y `size` limita el lote. `skip(page*size)` y `limit(size)` sirven en memoria; `totalElements` y `totalPages` forman parte del contrato. Se validan límites antes de operar. La evidencia correcta combina una entrada concreta con el status, las cabeceras y/o el cuerpo esperados, y deja claro qué capa aplica la regla.

Bloque 5 - Códigos de estado y buenas prácticas

T5.1 - Códigos de estado en GET

Los códigos de estado en GET son simples:

- GET de colección: siempre 200 OK. Si la colección está vacía, se devuelve [] con 200.
- GET individual existente: 200 OK con el recurso.
- GET individual inexistente: 404 Not Found.
- Parámetros inválidos: 400 Bad Request. Por ejemplo, si `page` es negativo o `size` es 0. Un error común es devolver 404 en un GET de colección cuando no hay elementos. Eso es incorrecto: la colección existe, solo está vacía. Otro error común es devolver 200 con null en un GET individual cuando el recurso no existe. Eso confunde al cliente: el código dice "éxito" pero el cuerpo está vacío. El código correcto es 404.

T5.2 - Buenas prácticas en GET

No modificar estado. GET debe ser seguro. No debe crear, actualizar ni eliminar nada. Devolver DTOs, no entidades. El controlador nunca ve entidades JPA. Solo DTOs. Usar `Optional` en el servicio. Cuando un recurso puede no existir, el servicio devuelve `Optional`. El controlador decide qué hacer con el vacío (normalmente, 404). Paginación por defecto. Las colecciones grandes deben paginarse. Aunque no se implemente al principio, preverlo. Filtros por query, no por ruta. `?curso=5º` es correcto; `/curso/5º` no lo es. No usar GET para operaciones con efectos. Los navegadores pueden pre-cargar URLs GET. Si tuvieran efectos, sería peligroso. Documentar los parámetros. En OpenAPI, documentar qué parámetros acepta cada endpoint.

T5.3 - Tests para GET

Un buen test de GET cubre varios casos:

- GET de colección con elementos: devuelve 200 y una lista no vacía.
- GET de colección vacía: devuelve 200 y [].
- GET de colección con filtro: devuelve solo los elementos que cumplen el filtro.
- GET de colección con ordenación: devuelve los elementos en el orden correcto.
- GET individual existente: devuelve 200 y el recurso.
- GET individual inexistente: devuelve 404.
- GET con parámetros inválidos: devuelve 400. En los tests del servicio, se mockea el repositorio y se verifica que el servicio filtra, ordena y pagina correctamente. En los tests del controlador, se usa MockMvc para simular peticiones y verificar códigos de estado y contenido. **Pregunta de cierre del bloque: ¿Por qué es importante probar el caso de colección vacía? ¿Qué error detecta?**

Actualización 2026 y vínculo con el proyecto

GET de colección válido devuelve 200 incluso vacío; GET individual ausente devuelve 404. Parámetros inválidos producen 400. Los tests deben cubrir filtros, límites, fechas, orden, metadatos y ausencia.

En el snapshot acumulativo de M3 esta idea no queda como teoría aislada: se refleja en el controlador, el servicio, los DTO y los tests que corresponden a este punto. La comprobación debe hacerse sobre el comportamiento HTTP observable y no sólo leyendo anotaciones.

Pregunta

¿Qué decisión de contrato de **códigos de estado y buenas prácticas** debería poder demostrar un test o una petición reproducible?

Respuesta razonada

GET de colección válido devuelve 200 incluso vacío; GET individual ausente devuelve 404. Parámetros inválidos producen 400. Los tests deben cubrir filtros, límites, fechas, orden, metadatos y ausencia. La evidencia correcta combina una entrada concreta con el status, las cabeceras y/o el cuerpo esperados, y deja claro qué capa aplica la regla.

Resumen del punto

- GET: consultar. Seguro, idempotente, cacheable.
- GET de colección: 200 con lista. Colección vacía → [] con 200.
- GET individual: 200 si existe, 404 si no.
- @PathVariable: extrae valores de la ruta. Obligatorio.
- @RequestParam: extrae valores de la query. Opcional con required = false.
- Filtrado: con .filter() en streams. Condicional si el parámetro es null.
- Ordenación: con .sorted() y un comparador.
- Paginación: con .skip() y .limit().
- Metadatos de paginación: content, page, size, totalElements, totalPages.
- Buenas prácticas: no modificar estado, DTOs, Optional, filtros en query.

Puente a la práctica

En la práctica 3.1 estas ideas se aplican sobre el mismo proyecto heredado de M2. Antes de continuar al siguiente punto se conserva la regresión anterior y se añade evidencia automatizada para el nuevo contrato.

Punto 3.2 - POST: crear recursos

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de: 1. Explicar qué significa crear un recurso y por qué POST es el método adecuado. 2. Recibir y deserializar el cuerpo de una petición con @RequestBody. 3. Devolver 201 Created con la cabecera Location y el recurso creado. 4. Validar los datos de entrada antes de crear el recurso. 5. Manejar conflictos (por ejemplo, duplicados) con 409 Conflict. 6. Escribir tests para endpoints POST. 7. Diagnosticar y resolver los errores más comunes al implementar POST.

Bloque 1 - Qué significa crear un recurso

T1.1 - El método POST y su semántica

POST es el método HTTP que significa “quiero crear un recurso nuevo con estos datos”. El cliente envía un cuerpo con los datos del recurso, y el servidor lo crea, le asigna un identificador y devuelve el recurso creado. POST tiene dos propiedades que lo distinguen de GET:

- No es seguro. Modifica el estado del servidor: crea un recurso. No se puede pre-cargar ni cachear.
- No es idempotente. Ejecutarlo N veces crea N recursos. Si haces cinco POST con los mismos datos, se crean cinco recursos (o se detecta el duplicado, según las reglas de negocio). Esa falta de idempotencia es importante. Si un POST falla por timeout y el cliente lo reintenta, puede crear un recurso duplicado. Para evitarlo, hay dos estrategias: usar un identificador de idempotencia (Idempotency-Key en una cabecera) o validar en el servidor que no exista un recurso con los mismos datos únicos (por ejemplo, el DNI). En nuestro caso, aplicaremos la segunda: si ya existe un alumno con el mismo DNI, el servidor devuelve 409 Conflict y no crea nada.

T1.2 - La URL y el cuerpo

En un POST, la URL apunta a la colección donde se quiere crear el recurso, no al recurso individual. El recurso aún no existe, así que no tiene ID.

POST /api/v1/alumnos

El cuerpo de la petición contiene los datos del recurso a crear. Los campos que el cliente envía son los que el servidor necesita para crear el recurso. Normalmente no se envía el ID: el servidor lo genera.

```
{
  "nombre": "María",
  "apellidos": "López Fernández",
  "dni": "11111111C",
  "fechaNacimiento": "2011-03-20",
  "curso": "4º Primaria"
}
```

El servidor recibe ese JSON, lo deserializa a un DTO, valida los datos, asigna un ID, guarda el recurso y devuelve el recurso creado (con el ID) al cliente.

T1.3 - Qué devuelve el servidor

La respuesta de un POST exitoso tiene tres partes: Código de estado 201 Created. No es 200. El 201 comunica específicamente que se ha creado un recurso. Es una distinción importante: 200 significa “éxito genérico”; 201 significa “éxito al crear”. Cabecera Location. Indica la URL del recurso creado. Por ejemplo: Location: /api/v1/alumnos/3. El cliente puede usar esa URL para consultar el recurso. Cuerpo con el recurso creado. Incluye todos los campos del recurso, incluido

el ID asignado. El cliente recibe confirmación de qué se ha creado y con qué ID. Un ejemplo de respuesta:

```
HTTP/1.1 201 Created
Location: /api/v1/alumnos/3
Content-Type: application/json

{
  "id": "3",
  "nombre": "María",
  "apellidos": "López Fernández",
  "dni": "11111111C",
  "fechaNacimiento": "2011-03-20",
  "curso": "4º Primaria"
}
```

Pregunta: ¿Por qué el servidor devuelve el recurso creado en lugar de solo un mensaje de confirmación?

Actualización 2026 y vínculo con el proyecto

POST sobre la colección solicita al servidor crear una nueva identidad. El cliente aporta datos de entrada, el servidor decide el identificador y devuelve la representación creada.

En el snapshot acumulativo de M3 esta idea no queda como teoría aislada: se refleja en el controlador, el servicio, los DTO y los tests que corresponden a este punto. La comprobación debe hacerse sobre el comportamiento HTTP observable y no sólo leyendo anotaciones.

Pregunta

¿Qué decisión de contrato de **qué significa crear un recurso** debería poder demostrar un test o una petición reproducible?

Respuesta razonada

POST sobre la colección solicita al servidor crear una nueva identidad. El cliente aporta datos de entrada, el servidor decide el identificador y devuelve la representación creada. La evidencia correcta combina una entrada concreta con el status, las cabeceras y/o el cuerpo esperados, y deja claro qué capa aplica la regla.

Bloque 2 - @RequestBody y deserialización

T2.1 - Qué hace @RequestBody

@RequestBody captura el cuerpo de la petición y lo deserializa a un objeto Java. Es la anotación que se usa en los parámetros de un método de controlador para recibir datos en POST, PUT y PATCH.

```
@PostMapping
public ResponseEntity<AlumnoDTO> crear(@RequestBody AlumnoDTO dto) {
    // ...
}
```

Cuando Spring MVC ve @RequestBody, hace lo siguiente: 1. Comprueba el Content-Type de la petición. Si es application/json, usa Jackson para deserializar. 2. Lee el cuerpo de la petición. 3. Deserializa el JSON a la clase indicada (AlumnoDTO). 4. Pasa el objeto al método. Si el cuerpo está vacío, Spring MVC lanza HttpMessageNotReadableException y devuelve un 400 Bad Request. Si el JSON está mal formado, también devuelve 400. Si el Content-Type no es application/json, devuelve 415 Unsupported Media Type.

T2.2 - Cómo Jackson deserializa

Jackson usa las mismas anotaciones que ya conocemos para deserializar:

- `@JsonProperty`: mapea un campo del JSON a un campo Java con nombre distinto.
- `@JsonFormat`: interpreta el formato de una fecha.
- `@JsonIgnore`: ignora un campo del JSON (no se deserializa). Para que Jackson pueda crear una instancia de la clase, esta necesita:
- Un constructor sin argumentos (público o accesible).
- Setters para los campos que se van a rellenar. Si falta el constructor sin argumentos, Jackson lanza `InvalidDefinitionException`: No default constructor found. Si falta un setter, el campo correspondiente no se rellena (se queda a null). Por defecto, Jackson ignora los campos desconocidos del JSON. Si el cliente envía un campo que no existe en el DTO, no falla: simplemente lo ignora. Esto es útil para evolucionar la API: si añades un campo al JSON en el futuro, los clientes antiguos no fallan.

T2.3 - Validación manual en el servicio

Aunque Spring MVC deserializa el JSON, no valida los datos. La validación de negocio (por ejemplo, “el DNI no puede estar duplicado”) va en el servicio. La validación de formato (por ejemplo, “el DNI debe tener 9 caracteres”) se puede hacer con Bean Validation, que veremos en el punto 3.5. En este punto, haremos validación manual en el servicio. Por ejemplo, comprobar que el DNI no esté duplicado:

```
public AlumnoDTO crear(AlumnoDTO dto) {  
    if (repositorio.existePorDni(dto.getDni())) {  
        throw new NegocioException("Ya existe un alumno con el DNI " + dto.getDni());  
    }  
    // ...  
}
```

El servicio comprueba la regla de negocio y, si se viola, lanza `NegocioException`. El controlador captura esa excepción (o un manejador global) y devuelve un 409 Conflict.

Pregunta: ¿Por qué la validación de negocio va en el servicio y no en el controlador?

Actualización 2026 y vínculo con el proyecto

Jackson transforma JSON a un DTO antes de entrar al controlador. JSON mal formado, tipos incompatibles o cuerpo ausente fallan en el borde HTTP y no deben confundirse con una regla de negocio.

En el snapshot acumulativo de M3 esta idea no queda como teoría aislada: se refleja en el controlador, el servicio, los DTO y los tests que corresponden a este punto. La comprobación debe hacerse sobre el comportamiento HTTP observable y no sólo leyendo anotaciones.

Pregunta

¿Qué decisión de contrato de **@requestbody y deserialización** debería poder demostrar un test o una petición reproducible?

Respuesta razonada

Jackson transforma JSON a un DTO antes de entrar al controlador. JSON mal formado, tipos incompatibles o cuerpo ausente fallan en el borde HTTP y no deben confundirse con una regla de negocio. La evidencia correcta combina una entrada concreta con el status, las cabeceras y/o el cuerpo esperados, y deja claro qué capa aplica la regla.

Bloque 3 - Códigos de estado y cabecera Location

T3.1 - 201 Created

El código 201 Created es el código de éxito para POST. Comunica específicamente que se ha creado un recurso. No es lo mismo que 200 OK, que es un éxito genérico. En Spring Boot, se construye con `ResponseEntity.created(location)`:

```
URI location = URI.create("/api/v1/alumnos/" + creado.getIdentificador());
return ResponseEntity.created(location).body(creado);
```

`ResponseEntity.created(location)` crea una respuesta 201 con la cabecera `Location` apuntando a la URI indicada. `.body(creado)` añade el recurso creado al cuerpo.

T3.2 - La cabecera Location

La cabecera `Location` indica la URL del recurso creado. Es una buena práctica incluirla, porque permite al cliente saber dónde consultar el recurso sin tener que construir la URL por su cuenta. La URI se construye a partir de la URL de la petición más el ID del recurso creado. Spring ofrece `ServletUriComponentsBuilder` para construirla de forma robusta:

```
URI location = ServletUriComponentsBuilder
    .fromCurrentRequest()
    .path("/{id}")
    .buildAndExpand(creado.getIdentificador())
    .toUri();
```

`fromCurrentRequest()` toma la URL actual (`/api/v1/alumnos`). `.path("/{id}")` añade la ruta del recurso individual. `.buildAndExpand(id)` reemplaza `{id}` con el ID del recurso creado. `.toUri()` construye la URI final. Esta forma es más robusta que concatenar strings, porque maneja correctamente los casos en los que la URL base tiene parámetros o contextos.

T3.3 - Conflictos y errores

Además del 201, un POST puede devolver otros códigos:

- 400 Bad Request. El cuerpo está mal formado, o los datos son inválidos.
- 409 Conflict. Ya existe un recurso con los mismos datos únicos (por ejemplo, el DNI).
- 415 Unsupported Media Type. El Content-Type no es `application/json`.
- 500 Internal Server Error. Error del servidor. En el controlador, estos errores se manejan con `@ExceptionHandler`. En el Módulo 5 veremos cómo centralizarlos con `@RestControllerAdvice`.

Pregunta de cierre del bloque: ¿Por qué es 409 y no 400 cuando hay un DNI duplicado? ¿Qué diferencia hay entre ambos?

Actualización 2026 y vínculo con el proyecto

Una creación correcta devuelve 201 y `Location` apuntando al recurso individual. `ServletUriComponentsBuilder` evita concatenar `host`, `puerto` o `context path` manualmente.

En el snapshot acumulativo de M3 esta idea no queda como teoría aislada: se refleja en el controlador, el servicio, los DTO y los tests que corresponden a este punto. La comprobación debe hacerse sobre el comportamiento HTTP observable y no sólo leyendo anotaciones.

Pregunta

¿Qué decisión de contrato de **códigos de estado y cabecera location** debería poder demostrar un test o una petición reproducible?

Respuesta razonada

Una creación correcta devuelve 201 y Location apuntando al recurso individual. ServletUriComponentsBuilder evita concatenar host, puerto o context path manualmente. La evidencia correcta combina una entrada concreta con el status, las cabeceras y/o el cuerpo esperados, y deja claro qué capa aplica la regla.

Bloque 4 - Validación de datos

T4.1 - Validación de formato vs validación de negocio

Hay dos tipos de validación: Validación de formato. Comprueba que los datos tienen el formato correcto: que un campo no esté vacío, que una fecha tenga el formato esperado, que un DNI tenga 9 caracteres, que un importe sea positivo. Es una validación estructural, independiente del estado del sistema. Validación de negocio. Comprueba que los datos cumplen las reglas del dominio: que el DNI no esté duplicado, que el alumno no esté ya matriculado en otro curso, que la beca no se solicite dos veces el mismo año. Es una validación contextual, que depende del estado del sistema. La validación de formato se puede hacer con Bean Validation (anotaciones como @NotBlank, @Size, @Email). La validación de negocio se hace en el servicio, con código explícito. En este punto haremos validación de negocio (DNI duplicado). La validación de formato con Bean Validation la veremos en el punto 3.5.

T4.2 - Validación de negocio en el servicio

La validación de negocio se implementa en el servicio, con código explícito:

```
public AlumnoDTO crear(AlumnoDTO dto) {
    if (repositorio.existePorDni(dto.getDni())) {
        throw new NegocioException("Ya existe un alumno con el DNI " + dto.getDni());
    }
    if (dto.getIdificador() == null || dto.getIdificador().isBlank()) {
        dto.setIdificador(String.valueOf(repositorio.contar() + 1));
    }
    return repositorio.guardar(dto);
}
```

El método comprueba las reglas de negocio y, si se violan, lanza NegocioException. Si no, asigna un ID y guarda el recurso. Este enfoque es explícito y fácil de testear. Cada regla de negocio es una línea de código. Si mañana se añade otra regla, se añade otra línea.

T4.3 - Manejo de la excepción en el controlador

El controlador captura la NegocioException con @ExceptionHandler y la convierte en un 409 Conflict:

```
@ExceptionHandler(NegocioException.class)
public ResponseEntity<Map<String, Object>> handleNegocio(NegocioException ex) {
    Map<String, Object> error = Map.of(
        "status", 409,
        "error", "Conflict",
        "message", ex.getMessage()
    );
    return ResponseEntity.status(HttpStatus.CONFLICT).body(error);
}
```

El manejador construye una respuesta con el código 409 y un JSON que describe el error. El cliente recibe un mensaje claro: "Ya existe un alumno con el DNI X". En el Módulo 5, este manejador se centralizará en una clase con @RestControllerAdvice, para no repetirlo en cada controlador.

Pregunta de cierre del bloque: ¿Por qué es mejor lanzar una excepción de negocio que devolver directamente un 409 desde el

servicio?

Actualización 2026 y vínculo con el proyecto

La forma del dato pertenece a Bean Validation y produce 400; la unicidad de DNI depende del estado del sistema y produce 409. Distinguir ambas categorías hace la API predecible.

En el snapshot acumulativo de M3 esta idea no queda como teoría aislada: se refleja en el controlador, el servicio, los DTO y los tests que corresponden a este punto. La comprobación debe hacerse sobre el comportamiento HTTP observable y no sólo leyendo anotaciones.

Pregunta

¿Qué decisión de contrato de **validación de datos** debería poder demostrar un test o una petición reproducible?

Respuesta razonada

La forma del dato pertenece a Bean Validation y produce 400; la unicidad de DNI depende del estado del sistema y produce 409. Distinguir ambas categorías hace la API predecible. La evidencia correcta combina una entrada concreta con el status, las cabeceras y/o el cuerpo esperados, y deja claro qué capa aplica la regla.

Bloque 5 - Buenas prácticas y tests**T5.1 - Buenas prácticas en POST**

No incluir el ID en el cuerpo. El ID lo genera el servidor. Si el cliente lo envía, se ignora o se rechaza. Devolver 201 con Location y el recurso creado. No devolver 200 con un mensaje. Validar los datos antes de crear. Tanto el formato (Bean Validation) como las reglas de negocio (servicio). Manejar duplicados con 409. No crear recursos duplicados silenciosamente. No usar POST para operaciones idempotentes. Si la operación se puede repetir sin efectos, usar PUT. Documentar el formato del cuerpo. En OpenAPI, describir los campos que acepta el POST. No exponer entidades. El POST recibe un DTO y devuelve un DTO. Nunca una entidad JPA.

T5.2 - Tests para POST

Un buen test de POST cubre varios casos:

- POST con datos válidos: devuelve 201, con Location y el recurso creado.
- POST con DNI duplicado: devuelve 409.
- POST con JSON mal formado: devuelve 400.
- POST sin Content-Type: devuelve 415. En el test del servicio, se mockea el repositorio y se verifica que el servicio crea el recurso y lanza excepción si hay duplicado. En el test del controlador, se usa MockMvc para simular la petición y verificar el código de estado, la cabecera Location y el cuerpo.

T5.3 - La cabecera Location en los tests

Verificar la cabecera Location en un test de POST es importante:

```
mockMvc.perform(post("/api/v1/alumnos")
    .contentType(MediaType.APPLICATION_JSON)
    .content("{\"nombre\":\"María\",...}"))
    .andExpect(status().isCreated())
```

```
.andExpect(header().string("Location", "/api/v1/alumnos/3"))  
.andExpect(jsonPath("$.id").value("3"));
```

`header().string("Location", "/api/v1/alumnos/3")` verifica que la cabecera Location tiene ese valor exacto. Si el controlador no la

incluye, el test falla. Verificar la cabecera Location garantiza que el cliente puede usar la URL del recurso creado. Es una parte importante del contrato de POST.

Pregunta de cierre del bloque: ¿Qué ventaja tiene que el test verifique la cabecera Location? ¿Qué error detecta?

Actualización 2026 y vínculo con el proyecto

El POST no es idempotente por definición general. Los tests verifican que un conflicto no escriba, que Location exista, que los errores de entrada sean 400 y que el recurso creado pueda consultarse.

En el snapshot acumulativo de M3 esta idea no queda como teoría aislada: se refleja en el controlador, el servicio, los DTO y los tests que corresponden a este punto. La comprobación debe hacerse sobre el comportamiento HTTP observable y no sólo leyendo anotaciones.

Pregunta

¿Qué decisión de contrato de **buenas prácticas y tests** debería poder demostrar un test o una petición reproducible?

Respuesta razonada

El POST no es idempotente por definición general. Los tests verifican que un conflicto no escriba, que Location exista, que los errores de entrada sean 400 y que el recurso creado pueda consultarse. La evidencia correcta combina una entrada concreta con el status, las cabeceras y/o el cuerpo esperados, y deja claro qué capa aplica la regla.

Resumen del punto

- POST: crear un recurso. No seguro, no idempotente.
- URL: apunta a la colección (/api/v1/alumnos). El ID lo genera el servidor.
- Cuerpo: contiene los datos del recurso a crear. No se envía el ID.
- Respuesta: 201 Created con cabecera Location y el recurso creado.
- @RequestBody: deserializa el cuerpo JSON a un DTO.
- Jackson: necesita constructor sin argumentos y setters. Ignora campos desconocidos.
- Validación: de formato (Bean Validation) y de negocio (servicio).
- Conflictos: 409 Conflict si hay duplicado.
- Buenas prácticas: no incluir ID, devolver 201 con Location, validar, manejar duplicados.
- Tests: verificar 201, Location, cuerpo, 409, 400, 415.

Puente a la práctica

En la práctica 3.2 estas ideas se aplican sobre el mismo proyecto heredado de M2. Antes de continuar al siguiente punto se conserva la regresión anterior y se añade evidencia automatizada para el nuevo contrato.

Punto 3.3 - PUT y PATCH: actualizar recursos

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de: 1. Explicar la diferencia entre PUT y PATCH y cuándo usar cada uno. 2. Implementar un endpoint PUT que reemplace completamente un recurso.

3. Implementar un endpoint PATCH que actualice parcialmente un recurso. 4. Manejar correctamente los códigos de estado en actualizaciones. 5. Aplicar validación de negocio en actualizaciones. 6. Escribir tests para endpoints PUT y PATCH. 7. Diagnosticar y resolver los errores más comunes al implementar actualizaciones.

Bloque 1 - Qué significa actualizar un recurso

T1.1 - Los dos sabores de la actualización

Actualizar un recurso significa modificar sus datos. Pero hay dos formas de hacerlo, y cada una tiene su propio método HTTP: PUT significa “reemplaza el recurso existente con esta representación completa”. El cliente envía todos los campos del recurso, y el servidor reemplaza el recurso existente con esos datos. Si un campo no se envía, se considera que debe quedar a null o a su valor por defecto. PATCH significa “modifica parcialmente el recurso con estos cambios”. El cliente envía solo los campos que quiere modificar, y el servidor actualiza esos campos y deja los demás como estaban. La diferencia es sutil pero importante. Un PUT es como reescribir todo el expediente desde cero; un PATCH es como añadir una diligencia al margen. Ambos modifican el recurso, pero de forma distinta. En una API REST, ambos métodos existen y tienen su lugar. La elección depende del caso de uso: si el cliente tiene todos los datos del recurso, usa PUT; si solo quiere cambiar un campo, usa PATCH.

T1.2 - PUT: reemplazo completo

PUT es idempotente: ejecutarlo N veces produce el mismo resultado que ejecutarlo una vez. Si envías el mismo PUT cinco veces, el recurso queda en el mismo estado las cinco veces. Eso es porque PUT reemplaza en lugar de acumular. La URL de un PUT apunta al recurso individual, no a la colección:

PUT /api/v1/alumnos/12345

El cuerpo contiene la representación completa del recurso:

```
{
  "id": "12345",
  "nombre": "Ana",
  "apellidos": "García López",
  "dni": "12345678A",
  "fechaNacimiento": "2010-05-12",
  "curso": "6º Primaria"
}
```

El servidor recibe ese JSON, busca el recurso con ese ID y lo reemplaza completamente. Si el recurso no existe, hay dos opciones:

devolver 404 (lo más común) o crear el recurso con ese ID (upsert). En este curso usaremos 404.

T1.3 - PATCH: actualización parcial

PATCH no es idempotente en general: aplicar el mismo PATCH dos veces puede producir resultados distintos. Por ejemplo, si un PATCH incrementa un contador, aplicarlo dos veces lo incrementa dos veces. La URL de un PATCH también apunta al recurso individual:

PATCH /api/v1/alumnos/12345

El cuerpo contiene solo los campos a modificar:

```
{
  "curso": "6º Primaria"
}
```

El servidor recibe ese JSON, busca el recurso, y actualiza solo los campos enviados. Los demás campos se quedan como estaban. El formato del cuerpo del PATCH no está estandarizado. El más común es enviar un objeto parcial con los campos a modificar, que es lo que usaremos. Otras aproximaciones usan JSON Patch (RFC 6902) con operaciones explícitas (add, remove, replace), pero son más complejas y menos comunes.

Pregunta: ¿Por qué PUT es idempotente y PATCH no? ¿Qué implica eso para los reintentos?

Actualización 2026 y vínculo con el proyecto

Actualizar exige preservar la identidad de la URL. PUT modela reemplazo completo y PATCH modificación parcial. El servicio comprueba existencia y reglas de negocio antes de persistir cambios.

En el snapshot acumulativo de M3 esta idea no queda como teoría aislada: se refleja en el controlador, el servicio, los DTO y los tests que corresponden a este punto. La comprobación debe hacerse sobre el comportamiento HTTP observable y no sólo leyendo anotaciones.

Pregunta

¿Qué decisión de contrato de **qué significa actualizar un recurso** debería poder demostrar un test o una petición reproducible?

Respuesta razonada

Actualizar exige preservar la identidad de la URL. PUT modela reemplazo completo y PATCH modificación parcial. El servicio comprueba existencia y reglas de negocio antes de persistir cambios. La evidencia correcta combina una entrada concreta con el status, las cabeceras y/o el cuerpo esperados, y deja claro qué capa aplica la regla.

Bloque 2 - PUT en profundidad

T2.1 - Semántica de PUT

PUT reemplaza todo el recurso. No es “actualiza los campos que envió”; es “toma este recurso completo y haz que el recurso en el servidor sea exactamente esto”. Eso significa que si el cliente envía un PUT con solo algunos campos, el servidor debería interpretar que los campos no enviados deben quedar a null o a su valor por defecto. En la práctica, muchos servidores son más permisivos y solo actualizan los campos enviados. Pero eso no es PUT puro: es más bien un PATCH encubierto. Para seguir las convenciones REST, el PUT debe recibir la representación completa del recurso. Si el recurso tiene 10 campos, el PUT debe incluir los 10. Si falta alguno, el servidor asume que ese campo debe quedar vacío.

T2.2 - Implementación de PUT en el servicio

En el servicio, el método actualizar recibe el ID y el DTO con los datos completos. Busca el recurso existente, y si lo encuentra, lo reemplaza:

```
public Optional<AlumnoDTO> actualizar(String id, AlumnoDTO dto) {  
    return repositorio.buscarPorId(id).map(existente -> {  
        dto.setIdentificador(id);  
        return repositorio.guardar(dto);  
    });  
}
```

repositorio.buscarPorId(id) busca el recurso existente. Devuelve Optional. .map(existente -> ...) aplica la transformación solo si el recurso existe. dto.setIdentificador(id) asegura que el ID del DTO coincide con el de la URL. Si el cliente envió un ID distinto en el cuerpo, se sobrescribe con

el de la URL. `repositorio.guardar(dto)` guarda el DTO, reemplazando el recurso existente. Si el recurso no existe, `buscarPorId` devuelve `Optional.empty()` y el map no se ejecuta. El método devuelve `Optional.empty()`, y el controlador lo traduce a un 404.

T2.3 - Implementación de PUT en el controlador

En el controlador, el método actualizar delega en el servicio y construye la respuesta:

```
@PutMapping("/{id}")
public ResponseEntity<AlumnoDTO> actualizar(
    @PathVariable String id,
    @RequestBody AlumnoDTO dto) {
    return service.actualizar(id, dto)
        .map(ResponseEntity::ok)
        .orElse(ResponseEntity.notFound().build());
}
```

- `@PathVariable String id` captura el ID de la URL.
- `@RequestBody AlumnoDTO dto` deserializa el cuerpo al DTO.

`service.actualizar(id, dto)` devuelve `Optional`. `.map(ResponseEntity::ok)` si el recurso existe, devuelve 200 con el recurso actualizado. `.orElse(ResponseEntity.notFound().build())` si no existe, devuelve 404.

Pregunta: ¿Qué pasa si el cliente envía un PUT con un ID en el cuerpo distinto al de la URL? ¿Cuál prevalece?

Actualización 2026 y vínculo con el proyecto

PUT debe ser idempotente para una misma representación. El id del cuerpo no gobierna la identidad: se impone el id de la ruta. Un recurso inexistente devuelve 404 en este contrato.

En el snapshot acumulativo de M3 esta idea no queda como teoría aislada: se refleja en el controlador, el servicio, los DTO y los tests que corresponden a este punto. La comprobación debe hacerse sobre el comportamiento HTTP observable y no sólo leyendo anotaciones.

Pregunta

¿Qué decisión de contrato de **put en profundidad** debería poder demostrar un test o una petición reproducible?

Respuesta razonada

PUT debe ser idempotente para una misma representación. El id del cuerpo no gobierna la identidad: se impone el id de la ruta. Un recurso inexistente devuelve 404 en este contrato. La evidencia correcta combina una entrada concreta con el status, las cabeceras y/o el cuerpo esperados, y deja claro qué capa aplica la regla.

Bloque 3 - PATCH en profundidad

T3.1 - Semántica de PATCH

PATCH modifica parcialmente el recurso. A diferencia de PUT, que reemplaza todo, PATCH solo toca los campos que se envían. Los campos no enviados conservan su valor actual. Eso tiene ventajas prácticas:

- Menos ancho de banda. Solo envías los campos que cambian.
- Menos errores. No tienes que recordar todos los campos del recurso.
- Más expresivo. Puedes modelar operaciones específicas (por ejemplo, "cambiar el estado"). Pero también tiene inconvenientes:

- Más complejo de implementar. El servidor tiene que decidir qué hacer con los campos no enviados.
- No es idempotente en general.
- Menos estandarizado. No hay un formato único para el cuerpo del PATCH. En la práctica, PATCH se usa para operaciones concretas: cambiar el estado de un expediente, actualizar un importe, marcar un recurso como eliminado.

T3.2 - Implementación de PATCH en el servicio

En el servicio, el método actualizarParcial recibe el ID y un Map<String, Object> con los campos a modificar:

```
public Optional<AlumnoDTO> actualizarParcial(String id, Map<String, Object> cambios) {
    return repositorio.buscarPorId(id).map(alumno -> {
        if (cambios.containsKey("nombre")) {
            alumno.setNombre((String) cambios.get("nombre"));
        }
        if (cambios.containsKey("apellidos")) {
            alumno.setApellidos((String) cambios.get("apellidos"));
        }
        if (cambios.containsKey("curso")) {
            alumno.setCurso((String) cambios.get("curso"));
        }
        return repositorio.guardar(alumno);
    });
}
```

Map<String, Object> cambios es el cuerpo del PATCH deserializado. Las claves son nombres de campos y los valores contienen los datos recibidos.

nuevos valores. cambios.containsKey("nombre") comprueba si el campo nombre está en el mapa. Si lo está, se actualiza. Si no, se deja como estaba. (String) cambios.get("nombre") obtiene el valor y lo convierte al tipo correcto. Object es el tipo genérico; hay que hacer un cast. repositorio.guardar(alumno) guarda el recurso modificado. Si el recurso no existe, buscarPorId devuelve Optional.empty() y el map no se ejecuta.

T3.3 - Implementación de PATCH en el controlador

En el controlador, el método actualizarParcial delega en el servicio:

```
@PatchMapping("/{id}")
public ResponseEntity<AlumnoDTO> actualizarParcial(
    @PathVariable String id,
    @RequestBody Map<String, Object> cambios) {
    return service.actualizarParcial(id, cambios)
        .map(ResponseEntity::ok)
        .orElse(ResponseEntity.notFound().build());
}
```

@PatchMapping("/{id}") mapea PATCH a /api/v1/alumnos/{id}. @RequestBody Map<String, Object> cambios deserializa el cuerpo a un mapa; Jackson convierte el JSON automáticamente.

service.actualizarParcial(id, cambios) devuelve Optional. .map(ResponseEntity::ok) si el recurso existe, devuelve 200 con el recurso actualizado. .orElse(ResponseEntity.notFound().build()) si no existe, devuelve 404.

Pregunta: ¿Por qué el PATCH usa Map en lugar de un DTO? ¿Qué ventaja tiene?

Actualización 2026 y vínculo con el proyecto

PATCH conserva campos no enviados. Un Map es útil didácticamente pero pierde tipado; por ello se limita a campos conocidos, se convierten fechas explícitamente y se rechazan valores incompatibles.

En el snapshot acumulativo de M3 esta idea no queda como teoría aislada: se refleja en el controlador, el servicio, los DTO y los tests que corresponden a este punto. La comprobación debe hacerse sobre el comportamiento HTTP observable y no sólo leyendo anotaciones.

Pregunta

¿Qué decisión de contrato de **patch en profundidad** debería poder demostrar un test o una petición reproducible?

Respuesta razonada

PATCH conserva campos no enviados. Un Map es útil didácticamente pero pierde tipado; por ello se limita a campos conocidos, se convierten fechas explícitamente y se rechazan valores incompatibles. La evidencia correcta combina una entrada concreta con el status, las cabeceras y/o el cuerpo esperados, y deja claro qué capa aplica la regla.

Bloque 4 - Códigos de estado y errores

T4.1 - Códigos de estado en PUT y PATCH

Los códigos de estado para PUT y PATCH son:

- 200 OK. Si se actualiza y se devuelve el recurso actualizado en el cuerpo.
- 204 No Content. Si se actualiza y no se devuelve nada.
- 404 Not Found. Si el recurso no existe.
- 400 Bad Request. Si los datos son inválidos o el cuerpo está mal formado.
- 409 Conflict. Si hay un conflicto con el estado actual (por ejemplo, DNI duplicado).
- 415 Unsupported Media Type. Si el Content-Type no es application/json. En la práctica, el 200 con el recurso actualizado es el más común. El 204 es más eficiente en ancho de banda, pero deja al cliente sin confirmación visual.

T4.2 - Validación en actualizaciones

La validación de negocio también se aplica en actualizaciones. Por ejemplo, si el cliente intenta cambiar el DNI a uno que ya existe en otro alumno, el servicio debe detectarlo y lanzar `NegocioException`.

```
public Optional<AlumnoDTO> actualizar(String id, AlumnoDTO dto) {
    if (repositorio.existePorDniYIdDistinto(dto.getDni(), id)) {
        throw new NegocioException("Ya existe otro alumno con el DNI " + dto.getDni());
    }
    return repositorio.buscarPorId(id).map(existente -> {
        dto.setIdentificador(id);
        return repositorio.guardar(dto);
    });
}
```

`existePorDniYIdDistinto(dni, id)` comprueba si existe otro alumno con ese DNI pero con un ID distinto. Si es así, hay conflicto. Este método hay que añadirlo al repositorio:

```
public boolean existePorDniYIdDistinto(String dni, String id) {
    return almacen.values().stream()
        .anyMatch(a -> a.getDni().equals(dni)
            && !a.getIdentificador().equals(id));
}
```

La condición es: el DNI coincide y el ID es distinto. Así no se considera duplicado si el DNI pertenece al mismo alumno que se está actualizando.

T4.3 - Manejo de errores en actualizaciones

El manejo de errores es el mismo que en POST: `NegocioException` se captura con `@ExceptionHandler` y se convierte en 409.

```
@ExceptionHandler(NegocioException.class)
public ResponseEntity<Map<String, Object>> handleNegocio(NegocioException ex) {
    Map<String, Object> error = Map.of(
        "status", 409,
        "error", "Conflict",
        "message", ex.getMessage()
    );
    return ResponseEntity.status(HttpStatus.CONFLICT).body(error);
}
```

En el Módulo 5, este manejador se centralizará en una clase con `@RestControllerAdvice`.

Pregunta de cierre del bloque: ¿Por qué es importante validar el DNI duplicado en una actualización? ¿Qué pasaría si no se

validara?

Actualización 2026 y vínculo con el proyecto

200 con cuerpo actualizado permite observar el resultado. 400 cubre formato/validación, 404 ausencia y 409 conflictos como DNI de otro alumno. Una fecha mal formateada no debe acabar en 500.

En el snapshot acumulativo de M3 esta idea no queda como teoría aislada: se refleja en el controlador, el servicio, los DTO y los tests que corresponden a este punto. La comprobación debe hacerse sobre el comportamiento HTTP observable y no sólo leyendo anotaciones.

Pregunta

¿Qué decisión de contrato de **códigos de estado y errores** debería poder demostrar un test o una petición reproducible?

Respuesta razonada

200 con cuerpo actualizado permite observar el resultado. 400 cubre formato/validación, 404 ausencia y 409 conflictos como DNI de otro alumno. Una fecha mal formateada no debe acabar en 500. La evidencia correcta combina una entrada concreta con el status, las cabeceras y/o el cuerpo esperados, y deja claro qué capa aplica la regla.

Bloque 5 - Buenas prácticas y tests

T5.1 - Buenas prácticas en PUT y PATCH

PUT:

- El ID va en la URL, no en el cuerpo. Si el cliente envía un ID en el cuerpo, se ignora o se rechaza.
- Validar que el recurso existe. Si no existe, devolver 404.
- Devolver el recurso actualizado o 204. Si el cliente necesita confirmación, devolver el recurso.
- No usar PUT para actualizaciones parciales. Para parciales, usar PATCH.
- Validar las reglas de negocio. Por ejemplo, DNI duplicado. PATCH:
- El ID va en la URL.

- Recibir solo los campos a modificar. No todos.
- Devolver el recurso actualizado o 204.
- Validar que el recurso existe.
- Documentar el formato del cuerpo. En OpenAPI, describir qué campos acepta el PATCH.

T5.2 - Tests para PUT y PATCH

Un buen test de PUT cubre:

- PUT con datos válidos: devuelve 200 con el recurso actualizado.
- PUT sobre recurso inexistente: devuelve 404.
- PUT con DNI duplicado: devuelve 409.
- PUT con JSON mal formado: devuelve 400. Un buen test de PATCH cubre:
- PATCH con un campo: devuelve 200 con el recurso actualizado (solo ese campo).
- PATCH con varios campos: devuelve 200 con todos actualizados.
- PATCH sobre recurso inexistente: devuelve 404.
- PATCH con un campo desconocido: el campo se ignora. En los tests del servicio, se mockea el repositorio y se verifica que el servicio actualiza correctamente. En los tests del controlador, se usa MockMvc para simular peticiones y verificar códigos de estado y contenido.

T5.3 - Verificar la inmutabilidad de campos no enviados en PATCH

Un test importante de PATCH es verificar que los campos no enviados no se modifican:

```
@Test
void actualizarParcial_debeMantenerCamposNoEnviados() {
    AlumnoDTO existente = new AlumnoDTO("1", "Ana", "García", "12345678A",
        LocalDate.of(2010, 5, 12), "5º Primaria");
    when(repositorio.buscarPorId("1")).thenReturn(Optional.of(existente));
    when(repositorio.guardar(any())).thenAnswer(inv -> inv.getArgument(0));

    Map<String, Object> cambios = Map.of("curso", "6º Primaria");
    Optional<AlumnoDTO> resultado = servicio.actualizarParcial("1", cambios);

    assertTrue(resultado.isPresent());
    assertEquals("Ana", resultado.get().getNombre()); // No cambia
    assertEquals("6º Primaria", resultado.get().getCurso()); // Cambia
}
```

El test verifica que nombre sigue siendo “Ana” (no se envió en el PATCH) y que curso ha cambiado a “6º Primaria” (sí se envió). Si el servicio modificara otros campos, el test fallaría.

Pregunta de cierre del bloque: ¿Por qué es importante verificar que los campos no enviados no se modifican en un PATCH?

Actualización 2026 y vínculo con el proyecto

Los tests de actualización deben demostrar idempotencia del PUT, preservación de campos en PATCH, unicidad excluyendo el propio recurso y ausencia de efectos laterales cuando falla la validación.

En el snapshot acumulativo de M3 esta idea no queda como teoría aislada: se refleja en el controlador, el servicio, los DTO y los tests que corresponden a este punto. La comprobación debe hacerse sobre el comportamiento HTTP observable y no sólo leyendo anotaciones.

Pregunta

¿Qué decisión de contrato de **buenas prácticas y tests** debería poder demostrar un test o una petición reproducible?

Respuesta razonada

Los tests de actualización deben demostrar idempotencia del PUT, preservación de campos en PATCH, unicidad excluyendo el propio recurso y ausencia de efectos laterales cuando falla la validación. La evidencia correcta combina una entrada concreta con el status, las cabeceras y/o el cuerpo esperados, y deja claro qué capa aplica la regla.

Resumen del punto

- PUT: reemplaza completamente un recurso. Idempotente.
- PATCH: modifica parcialmente un recurso. No idempotente.
- URL: apunta al recurso individual (/api/v1/alumnos/{id}).
- Cuerpo PUT: representación completa del recurso.
- Cuerpo PATCH: solo los campos a modificar.
- Códigos: 200, 204, 404, 400, 409, 415.
- Validación: de negocio en el servicio. DNI duplicado → 409.
- Buenas prácticas: ID en la URL, validar existencia, devolver recurso actualizado.
- Tests: verificar actualización, 404, 409, campos no modificados en PATCH.

Puente a la práctica

En la práctica 3.3 estas ideas se aplican sobre el mismo proyecto heredado de M2. Antes de continuar al siguiente punto se conserva la regresión anterior y se añade evidencia automatizada para el nuevo contrato.

Punto 3.4 - DELETE: eliminar recursos

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de: 1. Explicar qué significa eliminar un recurso y por qué DELETE es el método adecuado. 2. Diferenciar hard delete de soft delete y saber cuándo usar cada uno. 3. Implementar un endpoint DELETE que devuelva 204 No Content o 404 Not Found. 4. Entender la idempotencia de DELETE en la práctica. 5. Manejar dependencias entre recursos al eliminar. 6. Escribir tests para endpoints DELETE. 7. Diagnosticar y resolver los errores más comunes al implementar DELETE.

Bloque 1 - Qué significa eliminar un recurso

T1.1 - El método DELETE y su semántica

DELETE es el método HTTP que significa “quiero eliminar este recurso”. El cliente envía una petición a la URL del recurso individual, y el servidor lo elimina. No lleva cuerpo: la URL ya identifica qué se elimina. DELETE tiene dos propiedades que lo definen:

- No es seguro. Modifica el estado del servidor: elimina un recurso. No se puede pre-cargar ni cachear.
- Es idempotente. Ejecutarlo N veces produce el mismo resultado: el recurso deja de existir. Da igual cuántas veces lo llames; al final, el recurso no está. Esa idempotencia es importante. Si un DELETE falla por timeout y el cliente lo reintenta, el recurso se elimina (o ya estaba eliminado). No hay riesgo de duplicar efectos. Eso es distinto de POST, donde un reintento crea un recurso duplicado.

T1.2 - La URL de un DELETE

La URL de un DELETE apunta al recurso individual, no a la colección:

DELETE /api/v1/alumnos/12345

No se puede hacer un DELETE a la colección entera (DELETE /api/v1/alumnos) en una API REST bien diseñada. Eliminar toda una colección es una operación peligrosa y poco común. Si se necesita, se hace con un endpoint específico (DELETE /api/v1/alumnos/todos o POST /api/v1/alumnos/purgar), no con un DELETE genérico. El cliente no envía cuerpo. La URL ya contiene toda la información necesaria: el ID del recurso a eliminar.

T1.3 - Qué devuelve el servidor

La respuesta de un DELETE exitoso es simple: Código de estado 204 No Content. Comunica que la operación ha ido bien y que no hay nada que devolver. No es 200, porque 200 implicaría que hay un cuerpo. No es 202, porque la operación es síncrona. Es 204: éxito sin cuerpo. Si el recurso no existe, hay dos opciones:

- 404 Not Found. Es lo más común. El cliente sabe que el recurso no existía.
- 204 No Content. Algunos servidores devuelven 204 siempre, para no revelar si el recurso existía o no. Es una decisión de seguridad. En este curso usaremos 404 cuando el recurso no existe, porque es más informativo para el cliente. Y es idempotente: la segunda llamada devuelve 404, pero el estado final (recurso eliminado) es el mismo. Un ejemplo de respuesta:

HTTP/1.1 204 No Content

Date: Wed, 15 Jan 2025 10:00:00 GMT Sin cuerpo, sin Content-Type, solo la línea de estado y las cabeceras.

Pregunta: ¿Por qué DELETE devuelve 204 y no 200? ¿Qué diferencia hay entre ambos?

Actualización 2026 y vínculo con el proyecto

DELETE actúa sobre la identidad del recurso y es idempotente en efecto final. 204 indica eliminación aceptada sin cuerpo; 404 puede indicar que el recurso ya no existe.

En el snapshot acumulativo de M3 esta idea no queda como teoría aislada: se refleja en el controlador, el servicio, los DTO y los tests que corresponden a este punto. La comprobación debe hacerse sobre el comportamiento HTTP observable y no sólo leyendo anotaciones.

Pregunta

¿Qué decisión de contrato de **qué significa eliminar un recurso** debería poder demostrar un test o una petición reproducible?

Respuesta razonada

DELETE actúa sobre la identidad del recurso y es idempotente en efecto final. 204 indica eliminación aceptada sin cuerpo; 404 puede indicar que el recurso ya no existe. La evidencia correcta combina una entrada concreta con el status, las cabeceras y/o el cuerpo esperados, y deja claro qué capa aplica la regla.

Bloque 2 - Hard delete vs soft delete

T2.1 - Qué es hard delete

Hard delete significa eliminar el recurso físicamente: la fila se borra de la base de datos y desaparece para siempre. No hay forma de recuperarla (salvo restaurando una copia de seguridad). Es la forma más simple y la más común en APIs REST. Si un cliente elimina un recurso, el recurso deja de existir. Punto. La implementación es directa: el repositorio elimina el recurso del almacén.

```
public boolean eliminar(String id) {  
    return almacen.remove(id) != null;  
}
```

`almacen.remove(id)` elimina la entrada del mapa y devuelve el valor eliminado (o null si no existía). Comparamos con null para saber si se eliminó algo.

T2.2 - Qué es soft delete

Soft delete significa marcar el recurso como eliminado lógicamente, pero conservarlo en la base de datos. Se añade un campo eliminado (o activo, borrado, `deletedAt`) y se pone a true cuando se “elimina”. El recurso deja de aparecer en las consultas, pero sigue en la base de datos. Se usa cuando el dato tiene valor histórico o legal. Por ejemplo, en el Ministerio de Educación, un expediente eliminado puede que deba conservarse por razones de auditoría o porque la normativa exige mantener los registros durante un tiempo. La implementación es más compleja: hay que filtrar las consultas para que no devuelvan los eliminados, y hay que añadir un campo a la entidad.

```
public boolean eliminarLogico(String id) {  
    return almacen.computeIfPresent(id, (k, v) -> {  
        v.setEliminado(true);  
        return v;  
    }) != null;  
}
```

`computeIfPresent` ejecuta la función solo si la clave existe. Marca el recurso como eliminado y lo devuelve. En las consultas, se filtra:

```
public List<AlumnoDTO> listarTodos() {  
    return almacen.values().stream()  
        .filter(a -> !a.isEliminado())  
        .toList();  
}
```

T2.3 - Cuándo usar cada uno

La elección entre hard delete y soft delete depende del dominio: Hard delete:

- Datos temporales (sesiones, cachés, logs de un día).
- Datos sin valor histórico.
- Datos que el usuario espera que desaparezcan.
- Cuando la normativa no exige conservarlos. Soft delete:
- Datos con valor histórico (expedientes, facturas, resoluciones).
- Datos que la normativa exige conservar.
- Datos que pueden necesitarse para auditoría.
- Datos que pueden “restaurarse” (papelera de reciclaje). En el Ministerio de Educación, la mayoría de los datos administrativos son candidatos a soft delete. Un expediente eliminado puede que deba conservarse durante años. Por eso, en sistemas reales, se suele usar soft delete para los recursos principales. En este curso usaremos hard delete en el repositorio en memoria, porque es más sencillo. Pero conviene saber que en un sistema real se usaría soft delete para los datos sensibles. **Pregunta: ¿Qué ventaja tiene soft delete frente a hard delete en un sistema administrativo?**

Actualización 2026 y vínculo con el proyecto

Hard delete elimina físicamente; soft delete conserva el dato y lo excluye de consultas ordinarias. En sistemas administrativos, auditoría y conservación legal suelen favorecer borrado lógico.

En el snapshot acumulativo de M3 esta idea no queda como teoría aislada: se refleja en el controlador, el servicio, los DTO y los tests que corresponden a este punto. La comprobación debe hacerse sobre el comportamiento HTTP observable y no sólo leyendo anotaciones.

Pregunta

¿Qué decisión de contrato de **hard delete vs soft delete** debería poder demostrar un test o una petición reproducible?

Respuesta razonada

Hard delete elimina físicamente; soft delete conserva el dato y lo excluye de consultas ordinarias. En sistemas administrativos, auditoría y conservación legal suelen favorecer borrado lógico. La evidencia correcta combina una entrada concreta con el status, las cabeceras y/o el cuerpo esperados, y deja claro qué capa aplica la regla.

Bloque 3 - Implementación de DELETE

T3.1 - Implementación en el repositorio

El repositorio expone un método eliminar que devuelve un boolean indicando si se eliminó algo:

```
public boolean eliminar(String id) {  
    return almacen.remove(id) != null;  
}
```

almacen.remove(id) elimina la entrada del mapa y devuelve el valor eliminado. Si el ID no existe, devuelve null. != null convierte el resultado en un boolean: true si se eliminó algo, false si no. Este método es simple y claro. El repositorio no sabe nada de negocio: solo elimina.

T3.2 - Implementación en el servicio

El servicio delega en el repositorio:

```
public boolean eliminar(String id) {  
    return repositorio.eliminar(id);  
}
```

En este caso, el servicio no tiene lógica adicional. Podría tenerla: por ejemplo, comprobar que el recurso no tiene dependencias antes de eliminarlo. Pero para nuestro caso, delegar es suficiente. Si quisiéramos añadir lógica, sería algo así:

```
public boolean eliminar(String id) {  
    if (repositorio.tieneDependencias(id)) {  
        throw new NegocioException("No se puede eliminar: el recurso tiene dependencias");  
    }  
    return repositorio.eliminar(id);  
}
```

Esta comprobación iría en el servicio, no en el repositorio. El repositorio solo sabe eliminar; el servicio decide si se puede eliminar.

T3.3 - Implementación en el controlador

El controlador delega en el servicio y construye la respuesta:

```
@DeleteMapping("/{id}")  
public ResponseEntity<Void> eliminar(@PathVariable String id) {  
    return service.eliminar(id)  
        ? ResponseEntity.noContent().build()  
        : ResponseEntity.notFound().build();  
}
```

- @DeleteMapping("/{id}") mapea DELETE a /api/v1/alumnos/{id}.
- @PathVariable String id captura el ID de la URL.

`service.eliminar(id)` devuelve boolean. `? ResponseEntity.noContent().build() : ResponseEntity.notFound().build()` es un ternario: si se eliminó, devuelve 204; si no, 404. `ResponseEntity` indica que no hay cuerpo en la respuesta. `Void` es el tipo que se usa cuando no se devuelve nada.

Pregunta: ¿Por qué el controlador devuelve `ResponseEntity` y no `ResponseEntity`?

Actualización 2026 y vínculo con el proyecto

No siempre se puede borrar un recurso aislado. Si existen documentos asociados, el servicio debe decidir si bloquea, aplica cascada explícita o marca lógicamente. Esa coordinación no pertenece al repositorio.

En el snapshot acumulativo de M3 esta idea no queda como teoría aislada: se refleja en el controlador, el servicio, los DTO y los tests que corresponden a este punto. La comprobación debe hacerse sobre el comportamiento HTTP observable y no sólo leyendo anotaciones.

Pregunta

¿Qué decisión de contrato de **implementación de delete** debería poder demostrar un test o una petición reproducible?

Respuesta razonada

No siempre se puede borrar un recurso aislado. Si existen documentos asociados, el servicio debe decidir si bloquea, aplica cascada explícita o marca lógicamente. Esa coordinación no pertenece al repositorio. La evidencia correcta combina una entrada concreta con el status, las cabeceras y/o el cuerpo esperados, y deja claro qué capa aplica la regla.

Bloque 4 - Dependencias entre recursos

T4.1 - El problema de las dependencias

En un sistema real, los recursos tienen relaciones. Un alumno puede tener documentos asociados. Un expediente puede tener un solicitante. Un centro puede tener alumnos matriculados. Si eliminas un recurso que tiene dependencias, puedes dejar datos huérfanos o romper la integridad referencial. Por ejemplo, si eliminas un alumno que tiene documentos, los documentos quedan sin dueño. Hay varias estrategias para manejar esto:

- Cascada: al eliminar el recurso padre, se eliminan también los hijos.
- Restricción: no se permite eliminar el recurso padre si tiene hijos. Se devuelve un 409 Conflict.
- Anulación: los hijos se quedan, pero se marca que su padre ha sido eliminado (soft delete en cascada). La elección depende del dominio. En un sistema de gestión documental, probablemente se use restricción o soft delete en cascada.

T4.2 - Implementación de restricción

Si optamos por restricción, el servicio comprueba si hay dependencias antes de eliminar:

```
public boolean eliminar(String id) {
    if (documentoRepository.existePorAlumnoId(id)) {
        throw new NegocioException(
            "No se puede eliminar el alumno: tiene documentos asociados");
    }
    return repositorio.eliminar(id);
}
```

`documentoRepository.existePorAlumnoId(id)` comprueba si hay documentos asociados al alumno. `throw new NegocioException(...)` lanza la excepción si hay dependencias. El controlador captura la excepción con `@ExceptionHandler` y devuelve 409 Conflict.

T4.3 - Implementación de cascada

Si optamos por cascada, el servicio elimina primero los hijos y luego el padre:

```
public boolean eliminar(String id) {  
    documentoRepository.eliminarPorAlumnoId(id);  
    return repositorio.eliminar(id);  
}
```

documentoRepository.eliminarPorAlumnoId(id) elimina todos los documentos del alumno. repositorio.eliminar(id) elimina el alumno. Esta operación debería ser transaccional: si falla la eliminación del alumno, no deberían haberse eliminado los documentos. En el Módulo 4, cuando usemos JPA, veremos cómo gestionar transacciones con @Transactional.

Pregunta de cierre del bloque: ¿Qué estrategia usarías para eliminar un expediente que tiene documentos asociados? ¿Por qué?

Actualización 2026 y vínculo con el proyecto

La cascada debe ser deliberada y observable. Repetir DELETE no debe recrear efectos. Una primera llamada 204 y una segunda 404 siguen siendo compatibles con idempotencia de estado final.

En el snapshot acumulativo de M3 esta idea no queda como teoría aislada: se refleja en el controlador, el servicio, los DTO y los tests que corresponden a este punto. La comprobación debe hacerse sobre el comportamiento HTTP observable y no sólo leyendo anotaciones.

Pregunta

¿Qué decisión de contrato de **dependencias entre recursos** debería poder demostrar un test o una petición reproducible?

Respuesta razonada

La cascada debe ser deliberada y observable. Repetir DELETE no debe recrear efectos. Una primera llamada 204 y una segunda 404 siguen siendo compatibles con idempotencia de estado final. La evidencia correcta combina una entrada concreta con el status, las cabeceras y/o el cuerpo esperados, y deja claro qué capa aplica la regla.

Bloque 5 - Buenas prácticas y tests

T5.1 - Buenas prácticas en DELETE

Usar hard delete o soft delete según el dominio. No todo se puede eliminar físicamente. Devolver 204 No Content. No devolver 200 con un mensaje. Devolver 404 si el recurso no existe. O 204 si se quiere ser opaco. No usar GET para eliminar. GET /api/v1/alumnos/1/eliminar es incorrecto. Usar DELETE /api/v1/alumnos/1. Manejar dependencias. No dejar datos huérfanos. Documentar el comportamiento. En OpenAPI, indicar si es hard delete o soft delete. Ser consistente. Si un recurso usa soft delete, todos los recursos del mismo tipo deberían usarlo.

T5.2 - Tests para DELETE

Un buen test de DELETE cubre:

- DELETE de recurso existente: devuelve 204.
- DELETE de recurso inexistente: devuelve 404.
- DELETE dos veces: la segunda devuelve 404 (o 204, según la estrategia).
- DELETE con dependencias: devuelve 409 (si hay restricción).

- DELETE con soft delete: el recurso no aparece en las consultas posteriores. En los tests del servicio, se mockea el repositorio y se verifica que el servicio elimina correctamente. En los tests del controlador, se usa MockMvc para simular peticiones y verificar códigos de estado.

T5.3 - La idempotencia en los tests

Un test importante de DELETE es verificar la idempotencia:

```
@Test
void eliminar_debeDevolver204_primeravez() {
    when(repositorio.eliminar("1")).thenReturn(true);
    boolean resultado = servicio.eliminar("1");
    assertTrue(resultado);
}

@Test
void eliminar_debeDevolver404_segundaVez() {
    when(repositorio.eliminar("1")).thenReturn(false);
    boolean resultado = servicio.eliminar("1");
    assertFalse(resultado);
}
```

La primera llamada devuelve true (se eliminó). La segunda devuelve false (ya no existía). El estado final es el mismo: el recurso no existe. Eso es la idempotencia.

Pregunta de cierre del bloque: ¿Por qué DELETE es idempotente aunque la segunda llamada devuelva 404? ¿Qué importa: el

código o el estado final?

Actualización 2026 y vínculo con el proyecto

Los tests cubren recurso con dependencias, sin dependencias, cascada, soft delete, ocultación en listados y consulta posterior. Debe distinguirse 409 de negocio de un 500 inesperado.

En el snapshot acumulativo de M3 esta idea no queda como teoría aislada: se refleja en el controlador, el servicio, los DTO y los tests que corresponden a este punto. La comprobación debe hacerse sobre el comportamiento HTTP observable y no sólo leyendo anotaciones.

Pregunta

¿Qué decisión de contrato de **buenas prácticas y tests** debería poder demostrar un test o una petición reproducible?

Respuesta razonada

Los tests cubren recurso con dependencias, sin dependencias, cascada, soft delete, ocultación en listados y consulta posterior. Debe distinguirse 409 de negocio de un 500 inesperado. La evidencia correcta combina una entrada concreta con el status, las cabeceras y/o el cuerpo esperados, y deja claro qué capa aplica la regla.

Resumen del punto

- DELETE: eliminar un recurso. No seguro, idempotente.
- URL: apunta al recurso individual (/api/v1/alumnos/{id}).
- Cuerpo: no lleva.
- Respuesta: 204 No Content si se elimina, 404 si no existe.
- Hard delete: elimina físicamente. Soft delete: marca como eliminado.
- Dependencias: cascada, restricción o anulación.

- Buenas prácticas: no usar GET, manejar dependencias, documentar.
- Tests: verificar 204, 404, idempotencia.

Puente a la práctica

En la práctica 3.4 estas ideas se aplican sobre el mismo proyecto heredado de M2. Antes de continuar al siguiente punto se conserva la regresión anterior y se añade evidencia automatizada para el nuevo contrato.

Punto 3.5 - DTOs avanzados y validaciones

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de: 1. Diferenciar DTOs de entrada y DTOs de salida y saber por qué separarlos. 2. Crear DTOs anidados para representar estructuras complejas. 3. Aplicar Bean Validation con anotaciones como @NotBlank, @Size, @Email, @PositiveOrZero. 4. Activar la validación con @Valid en el controlador. 5. Manejar los errores de validación con MethodArgumentNotValidException. 6. Escribir tests para validaciones. 7. Diagnosticar y resolver los errores más comunes al validar datos.

Bloque 1 - DTOs de entrada y de salida

T1.1 - Por qué separar DTOs de entrada y de salida

Hasta ahora hemos usado un solo DTO para todo: el mismo AlumnoDTO sirve para recibir datos en un POST, para devolver datos en un GET y para actualizar en un PUT. Eso funciona en proyectos pequeños, pero tiene problemas. El primer problema es la seguridad. Un DTO de entrada puede tener campos que el cliente no debería poder establecer. Por ejemplo, el campo id lo genera el servidor; si el cliente lo envía en un POST, se ignora. Pero si el DTO es el mismo, el cliente ve ese campo y puede pensar que tiene control sobre él. Peor aún: si hay un campo activo o eliminado, el cliente podría intentar establecerlo. El segundo problema es la validación. Un DTO de entrada necesita validaciones (@NotBlank, @Size) que un DTO de salida no necesita. El DTO de salida ya contiene datos que el servidor ha validado; no hace falta volver a validarlos. El tercer problema es la claridad. Un DTO de entrada representa lo que el cliente envía; un DTO de salida representa lo que el servidor devuelve. Si son el mismo, el contrato es ambiguo: ¿qué campos son obligatorios? ¿cuáles son de solo lectura? Por eso, en proyectos profesionales se separan:

- DTO de entrada (AlumnoRequestDTO o CrearAlumnoDTO): los campos que el cliente envía. Lleva las validaciones.
- DTO de salida (AlumnoResponseDTO o AlumnoDTO): los campos que el servidor devuelve. Sin validaciones.

T1.2 - DTO de entrada

Un DTO de entrada contiene solo los campos que el cliente puede enviar. No incluye el ID (lo genera el servidor), ni campos de auditoría (fecha de creación, usuario que creó), ni campos calculados.

```
public class AlumnoRequestDTO {  
  
    @NotBlank(message = "El nombre es obligatorio")  
    @Size(max = 100, message = "El nombre no puede superar 100 caracteres")  
    private String nombre;  
  
    @NotBlank(message = "Los apellidos son obligatorios")  
    private String apellidos;  
  
    @NotBlank(message = "El DNI es obligatorio")
```

```

@Size(min = 9, max = 9, message = "El DNI debe tener 9 caracteres")
private String dni;

@NotNull(message = "La fecha de nacimiento es obligatoria")
@JsonFormat(pattern = "yyyy-MM-dd")
private LocalDate fechaNacimiento;

@NotBlank(message = "El curso es obligatorio")
private String curso;

// constructores, getters y setters
}

```

Cada campo tiene las anotaciones de validación que corresponden. El cliente no puede enviar un id porque no existe en el DTO. Si lo envía, Jackson lo ignora.

T1.3 - DTO de salida

Un DTO de salida contiene todos los campos que el servidor devuelve, incluidos los que el cliente no puede establecer: ID, fecha de creación, estado, etc.

```

public class AlumnoResponseDTO {

    @JsonProperty("id")
    private String identificador;

    private String nombre;
    private String apellidos;
    private String dni;

    @JsonFormat(pattern = "yyyy-MM-dd")
    private LocalDate fechaNacimiento;

    private String curso;
    private Boolean activo;

    // constructores, getters y setters
}

```

El DTO de salida no tiene validaciones porque los datos ya están validados en el servidor. Y puede tener campos que el DTO de entrada no tiene (como id o activo).

Pregunta: ¿Por qué el DTO de salida no necesita validaciones? ¿Qué valida el servidor antes de devolverlo?

Actualización 2026 y vínculo con el proyecto

Separar request y response impide que el cliente controle campos del servidor. El request expresa datos aceptados; el response puede exponer id, estado derivado y campos de lectura sin mezclar constraints de entrada.

En el snapshot acumulativo de M3 esta idea no queda como teoría aislada: se refleja en el controlador, el servicio, los DTO y los tests que corresponden a este punto. La comprobación debe hacerse sobre el comportamiento HTTP observable y no sólo leyendo anotaciones.

Pregunta

¿Qué decisión de contrato de **dtos de entrada y de salida** debería poder demostrar un test o una petición reproducible?

Respuesta razonada

Separar request y response impide que el cliente controle campos del servidor. El request expresa datos aceptados; el response puede exponer id, estado derivado y campos de lectura sin mezclar constraints de entrada. La evidencia correcta combina una entrada concreta con el status, las cabeceras y/o el cuerpo esperados, y deja claro qué capa aplica la regla.

Bloque 2 - DTOs anidados

T2.1 - Qué es un DTO anidado

Un DTO anidado es un DTO que contiene otros DTOs como campos. Se usa cuando un recurso tiene sub-recursos o cuando una entidad tiene relaciones con otras. Por ejemplo, un ExpedienteDTO puede tener un SolicitanteDTO anidado:

```
public class ExpedienteDTO {  
  
    @JsonProperty("id")  
    private String identificador;  
  
    private SolicitanteDTO solicitante;  
  
    private String estado;  
    private String tipo;  
  
    @JsonFormat(pattern = "yyyy-MM-dd")  
    private LocalDate fechaSolicitud;  
  
    private Double importe;  
  
    // getters y setters  
}
```

Y el SolicitanteDTO:

```
public class SolicitanteDTO {  
  
    private String nombre;  
    private String apellidos;  
  
    @JsonProperty("dni")  
    private String documentoIdentidad;  
  
    // getters y setters  
}
```

Cuando Jackson serializa el ExpedienteDTO, incluye el SolicitanteDTO como un objeto JSON anidado:

```
{  
  "id": "1",  
  "solicitante": {  
    "nombre": "Ana",  
    "apellidos": "García",  
    "dni": "12345678A"  
  },  
  "estado": "EN_TRAMITE",  
  "tipo": "BECA",  
  "fechaSolicitud": "2025-01-15",  
}
```

```
"importe": 1500.0
}
```

T2.2 - Cuándo usar DTOs anidados

Los DTOs anidados se usan cuando:

- El recurso tiene sub-recursos. Un expediente tiene un solicitante, unos documentos, un historial.
- El recurso tiene relaciones. Un alumno pertenece a un curso, que pertenece a un centro.
- Quieres agrupar campos relacionados. En lugar de tener nombreSolicitante, apellidosSolicitante, dniSolicitante, se agrupan en un SolicitanteDTO. La ventaja es la claridad: el JSON refleja la estructura del dominio. El cliente ve que el expediente tiene un solicitante, y el solicitante tiene nombre, apellidos y DNI. El inconveniente es la complejidad: hay que crear más clases y gestionar las transformaciones entre entidades y DTOs. Pero para dominios con relaciones, merece la pena.

T2.3 - DTOs anidados con listas

Un DTO puede contener una lista de otros DTOs. Por ejemplo, un ExpedienteDTO con una lista de documentos:

```
public class ExpedienteDTO {

    @JsonProperty("id")
    private String identificador;

    private String estado;
    private String tipo;

    private List<DocumentoDTO> documentos;

    // getters y setters
}
```

Y el DocumentoDTO:

```
public class DocumentoDTO {

    private String nombre;
    private String tipo;
    private Long tamano;

    // getters y setters
}
```

Jackson serializa la lista como un array JSON:

```
{
  "id": "1",
  "estado": "EN_TRAMITE",
  "tipo": "BECA",
  "documentos": [
    {"nombre": "DNI.pdf", "tipo": "application/pdf", "tamano": 12345},
    {"nombre": "Notas.pdf", "tipo": "application/pdf", "tamano": 67890}
  ]
}
```

Pregunta: ¿Qué ventaja tiene agrupar campos relacionados en un DTO anidado en lugar de tenerlos todos en un solo DTO?

Actualización 2026 y vínculo con el proyecto

Los objetos anidados se validan con @Valid. Un ExpedienteRequestDTO puede contener SolicitanteDTO: no basta validar el contenedor; hay que propagar la validación al objeto hijo.

En el snapshot acumulativo de M3 esta idea no queda como teoría aislada: se refleja en el controlador, el servicio, los DTO y los tests que corresponden a este punto. La comprobación debe hacerse sobre el comportamiento HTTP observable y no sólo leyendo anotaciones.

Pregunta

¿Qué decisión de contrato de **dtos anidados** debería poder demostrar un test o una petición reproducible?

Respuesta razonada

Los objetos anidados se validan con @Valid. Un ExpedienteRequestDTO puede contener SolicitanteDTO: no basta validar el contenedor; hay que propagar la validación al objeto hijo. La evidencia correcta combina una entrada concreta con el status, las cabeceras y/o el cuerpo esperados, y deja claro qué capa aplica la regla.

Bloque 3 - Bean Validation

T3.1 - Qué es Bean Validation

Bean Validation es un estándar de Java para validar objetos. Define un conjunto de anotaciones (@NotNull, @NotBlank, @Size, @Email, etc.) que se aplican a los campos de una clase. Un validador lee esas anotaciones y comprueba que los valores cumplen las restricciones. En Spring Boot, Bean Validation se incluye dentro de spring-boot-starter-validation. Si no lo tienes, se añade al pom.xml:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

La implementación por defecto es Hibernate Validator, que es la implementación de referencia del estándar. Bean Validation se usa en los DTOs de entrada, no en los de salida. Los DTOs de salida ya contienen datos validados. Y se usa en los controladores, con la anotación @Valid, que activa la validación antes de ejecutar el método.

T3.2 - Anotaciones de validación

Las anotaciones más usadas son:

- @NotNull: el campo no puede ser null. Pero puede ser vacío.
- @NotBlank: el campo no puede ser null, ni vacío, ni solo espacios. Se aplica a strings.
- @NotEmpty: el campo no puede ser null ni vacío. Se aplica a strings, colecciones y mapas.
- @Size(min, max): el campo debe tener entre min y max caracteres (para strings) o elementos (para colecciones).
- @Min(value): el campo debe ser mayor o igual que value.
- @Max(value): el campo debe ser menor o igual que value.
- @Positive: el campo debe ser positivo (mayor que 0).
- @PositiveOrZero: el campo debe ser positivo o cero.
- @Email: el campo debe ser un email válido.
- @Pattern(regexp): el campo debe cumplir una expresión regular.
- @Past: el campo debe ser una fecha pasada.
- @Future: el campo debe ser una fecha futura. Todas ellas aceptan un atributo message para personalizar el mensaje de error:

```
@NotBlank(message = "El nombre es obligatorio")
private String nombre;
```

Si no se especifica message, se usa un mensaje por defecto (en inglés, normalmente).

T3.3 - Activar la validación con @Valid

Para que Bean Validation se ejecute, hay que anotar el parámetro del controlador con @Valid:

```
@PostMapping
public ResponseEntity<AlumnoResponseDTO> crear(
    @Valid @RequestBody AlumnoRequestDTO dto) {
    // ...
}
```

@Valid indica a Spring MVC que valide dto antes de ejecutar el método. Si hay errores de validación, Spring MVC construye la respuesta de error antes de entrar en la lógica de aplicación.

lanza MethodArgumentNotValidException y no ejecuta el método. El controlador (o un manejador global) captura esa excepción y devuelve un 400 con los detalles. Sin @Valid, la validación no se ejecuta. El objeto llega al método con los valores que el cliente haya enviado, sin comprobar.

Pregunta: ¿Qué pasa si olvidas @Valid en un parámetro que debería validarse?

Actualización 2026 y vínculo con el proyecto

@NotBlank, @Size, @NotNull y @Past convierten reglas de forma en un contrato declarativo. La dependencia spring-boot-starter-validation integra Jakarta Validation con Spring MVC.

En el snapshot acumulativo de M3 esta idea no queda como teoría aislada: se refleja en el controlador, el servicio, los DTO y los tests que corresponden a este punto. La comprobación debe hacerse sobre el comportamiento HTTP observable y no sólo leyendo anotaciones.

Pregunta

¿Qué decisión de contrato de **bean validation** debería poder demostrar un test o una petición reproducible?

Respuesta razonada

@NotBlank, @Size, @NotNull y @Past convierten reglas de forma en un contrato declarativo. La dependencia spring-boot-starter-validation integra Jakarta Validation con Spring MVC. La evidencia correcta combina una entrada concreta con el status, las cabeceras y/o el cuerpo esperados, y deja claro qué capa aplica la regla.

Bloque 4 - Manejo de errores de validación

T4.1 - Qué es MethodArgumentNotValidException

Cuando @Valid detecta errores de validación, Spring MVC lanza MethodArgumentNotValidException. Es una excepción que contiene la lista de errores: qué campo ha fallado y por qué. Si no se captura, Spring Boot devuelve un 500 Internal Server Error. Pero no es un error del servidor: es un error del cliente. El código correcto es 400 Bad Request. Para capturarla, se usa un @ExceptionHandler:

```
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<Map<String, Object>> handleValidation(
    MethodArgumentNotValidException ex) {
    // ...
}
```

El método recibe la excepción y construye la respuesta.

T4.2 - Extraer los errores

La excepción `MethodArgumentNotValidException` tiene un método `getBindingResult()` que devuelve un `BindingResult`. Ese objeto contiene la lista de errores de campo:

```
List<Map<String, String>> errores = ex.getBindingResult()
    .getFieldErrors()
    .stream()
    .map(e -> Map.of(
        "field", e.getField(),
        "message", e.getDefaultMessage()))
    .toList();
```

`getFieldErrors()` devuelve la lista de errores por campo. `e.getField()` devuelve el nombre del campo que ha fallado. `e.getDefaultMessage()` devuelve el mensaje de error. El resultado es una lista de mapas, cada uno con el campo y el mensaje:

```
[
  {"field": "nombre", "message": "El nombre es obligatorio"},
  {"field": "dni", "message": "El DNI debe tener 9 caracteres"}
]
```

T4.3 - Construir la respuesta de error

Con la lista de errores, se construye la respuesta:

```
Map<String, Object> respuesta = Map.of(
    "timestamp", Instant.now().toString(),
    "status", 400,
    "error", "Bad Request",
    "message", "Errores de validación",
    "errors", errores
);
return ResponseEntity.badRequest().body(respuesta);
```

La respuesta incluye:

- timestamp: el momento del error.
- status: 400.
- error: "Bad Request".
- message: un mensaje genérico.
- errors: la lista de errores por campo. El cliente recibe un JSON claro con todos los errores. Puede mostrarlos al usuario o corregirlos automáticamente. **Pregunta de cierre del bloque: ¿Por qué es importante devolver la lista de errores por campo en lugar de un solo mensaje genérico?**

genérico?

Actualización 2026 y vínculo con el proyecto

`MethodArgumentNotValidException` contiene errores por campo. Una respuesta 400 estable debe exponer campo y mensaje sin filtrar detalles internos del framework. El orden de errores no debe asumirse si no se garantiza.

En el snapshot acumulativo de M3 esta idea no queda como teoría aislada: se refleja en el controlador, el servicio, los DTO y los tests que corresponden a este punto. La comprobación debe hacerse sobre el comportamiento HTTP observable y no sólo leyendo anotaciones.

Pregunta

¿Qué decisión de contrato de **manejo de errores de validación** debería poder demostrar un test o una petición reproducible?

Respuesta razonada

MethodArgumentNotValidException contiene errores por campo. Una respuesta 400 estable debe exponer campo y mensaje sin filtrar detalles internos del framework. El orden de errores no debe asumirse si no se garantiza. La evidencia correcta combina una entrada concreta con el status, las cabeceras y/o el cuerpo esperados, y deja claro qué capa aplica la regla.

Bloque 5 - Buenas prácticas y tests

T5.1 - Buenas prácticas en DTOs y validaciones

Separar DTOs de entrada y de salida. El de entrada tiene validaciones; el de salida, no. Usar Bean Validation para validar el formato. @NotBlank, @Size, @Email, etc. Dejar la validación de negocio en el servicio. Bean Validation no sabe si un DNI está duplicado. Manejar los errores de validación con @ExceptionHandler. Devolver 400 con la lista de errores. Documentar las validaciones en OpenAPI. El cliente debe saber qué campos son obligatorios. No usar el mismo DTO para todo. Si el DTO de entrada y el de salida son distintos, sepáralos. No validar en el controlador. La validación de formato va en el DTO con anotaciones; la de negocio, en el servicio.

T5.2 - Tests para validaciones

Un buen test de validación cubre:

- POST con datos válidos: devuelve 201.
- POST con un campo obligatorio vacío: devuelve 400 con el error del campo.
- POST con un campo demasiado largo: devuelve 400.
- POST con un formato inválido: devuelve 400.
- POST con varios errores: devuelve 400 con todos los errores. En los tests del controlador, se usa MockMvc para enviar peticiones con datos inválidos y verificar que se devuelve 400 con los errores esperados. En los tests del servicio, la validación de Bean Validation no se ejecuta, porque el servicio no tiene @Valid. Los tests del servicio se centran en la validación de negocio.

T5.3 - Verificar los mensajes de error en los tests

Un test importante es verificar que el mensaje de error es el esperado:

```
mockMvc.perform(post("/api/v1/alumnos")
    .contentType(MediaType.APPLICATION_JSON)
    .content("""
        {"nombre": "", "apellidos": "López",
         "dni": "123", "fechaNacimiento": "2011-03-20",
         "curso": "4º"}
        """))
    .andExpect(status().isBadRequest())
    .andExpect(jsonPath("$.errors[?(@.field=='nombre')].message")
        .value("El nombre es obligatorio"))
    .andExpect(jsonPath("$.errors[?(@.field=='dni')].message")
        .value("El DNI debe tener 9 caracteres"));
```

jsonPath("\$.errors[?(@.field=='nombre')].message") navega por el array de errores, selecciona el elemento cuyo field es nombre y extrae su mensaje.

su message. Verificar los mensajes garantiza que el cliente recibe información útil para corregir los datos.

Pregunta de cierre del bloque: ¿Por qué es importante que los mensajes de error sean específicos y no genéricos?

Actualización 2026 y vínculo con el proyecto

Las constraints deben probarse con casos límite y mensajes comprensibles. La validación de forma no sustituye reglas de negocio; DNI con nueve caracteres puede seguir ser duplicado y devolver 409.

En el snapshot acumulativo de M3 esta idea no queda como teoría aislada: se refleja en el controlador, el servicio, los DTO y los tests que corresponden a este punto. La comprobación debe hacerse sobre el comportamiento HTTP observable y no sólo leyendo anotaciones.

Pregunta

¿Qué decisión de contrato de **buenas prácticas y tests** debería poder demostrar un test o una petición reproducible?

Respuesta razonada

Las constraints deben probarse con casos límite y mensajes comprensibles. La validación de forma no sustituye reglas de negocio; DNI con nueve caracteres puede seguir ser duplicado y devolver 409. La evidencia correcta combina una entrada concreta con el status, las cabeceras y/o el cuerpo esperados, y deja claro qué capa aplica la regla.

Resumen del punto

- DTOs de entrada y salida: separados por seguridad, validación y claridad.
- DTO de entrada: solo campos que el cliente puede enviar. Lleva validaciones.
- DTO de salida: todos los campos que el servidor devuelve. Sin validaciones.
- DTOs anidados: para representar sub-recursos y relaciones.
- Bean Validation: anotaciones para validar formato. @NotBlank, @Size, @Email, etc.
- @Valid: activa la validación en el controlador.
- MethodArgumentNotValidException: captura los errores de validación.
- Manejo: devolver 400 con la lista de errores por campo.
- Buenas prácticas: separar DTOs, validar formato en el DTO, validar negocio en el servicio.
- Tests: verificar 400 y los mensajes de error.

Puente a la práctica

En la práctica 3.5 estas ideas se aplican sobre el mismo proyecto heredado de M2. Antes de continuar al siguiente punto se conserva la regresión anterior y se añade evidencia automatizada para el nuevo contrato.

Punto 3.6 - Parámetros, cabeceras y Postman

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de: 1. Usar @RequestParam con valores por defecto, listas y mapas. 2. Extraer cabeceras HTTP con @RequestHeader. 3. Leer cookies con @CookieValue. 4. Conocer las cabeceras HTTP más útiles en una API REST. 5. Instalar y usar Postman para probar APIs de forma gráfica. 6. Organizar peticiones en colecciones y entornos de Postman. 7. Diagnosticar y resolver los errores más comunes al trabajar con parámetros y cabeceras.

Bloque 1 - @RequestParam en profundidad

T1.1 - Recordatorio y variantes

@RequestParam extrae un valor de la query string de la URL y lo inyecta como parámetro del método. Ya lo hemos usado en puntos anteriores, pero tiene variantes que conviene conocer. La forma básica es:

```
@GetMapping
public List<AlumnoDTO> listar(@RequestParam String curso) {
    // ...
}
```

Por defecto, @RequestParam es obligatorio: si el cliente no envía el parámetro, Spring MVC devuelve un 400 Bad Request. Para hacerlo opcional, se usa required = false:

```
@RequestParam(required = false) String curso
```

Para darle un valor por defecto, se usa defaultValue:

```
@RequestParam(defaultValue = "0") int page
```

Si el cliente no envía page, el parámetro valdrá 0. Si lo envía, valdrá lo que envíe. defaultValue implica required = false: no hace falta ponerlo.

Pregunta: ¿Qué diferencia hay entre required = false y defaultValue? ¿Cuándo usarías cada uno?

T1.2 - Listas y mapas como parámetros

@RequestParam también puede capturar varios valores del mismo parámetro en una lista:

```
@GetMapping
public List<AlumnoDTO> listar(@RequestParam List<String> curso) {
    // ...
}
```

Si el cliente envía /api/v1/alumnos?curso=5º&curso=6º, la lista curso contendrá ["5º", "6º"]. Si no envía el parámetro, la lista estará vacía (no será null). También se puede capturar todos los query parameters en un mapa:

```
@GetMapping
public List<AlumnoDTO> listar(@RequestParam Map<String, String> filtros) {
    // ...
}
```

Si el cliente envía /api/v1/alumnos?curso=5º&dni=12345678A, el mapa contendrá {"curso": "5º", "dni": "12345678A"}. Es útil cuando no sabes de antemano qué filtros va a enviar el cliente.

Pregunta: ¿Qué ventaja tiene capturar todos los query parameters en un Map en lugar de declarar cada uno?

T1.3 - @RequestParam con tipos no String

Spring MVC convierte automáticamente los query parameters al tipo que declares. Si el parámetro es int, Spring convierte el string a int. Si es LocalDate, Spring lo convierte usando el formato configurado (por defecto, ISO 8601: yyyy-MM-dd).

```
@GetMapping
public List<AlumnoDTO> listar(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(required = false)
    @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate fechaDesde) {
```

```
// ...
}
```

`@DateTimeFormat(iso = DateTimeFormat.ISO.DATE)` indica a Spring que use ISO-8601 al convertir el parámetro recibido a `LocalDate`.

Sin esta anotación, Spring usaría el formato por defecto, que puede no coincidir. Si la conversión falla (por ejemplo, el cliente envía `page=abc`), Spring MVC devuelve un 400 Bad Request con un mensaje que describe el error.

Pregunta de cierre del bloque: ¿Qué pasa si el cliente envía un page no numérico? ¿Y si envía una fecha con formato incorrecto?

Actualización 2026 y vínculo con el proyecto

Los query parameters permiten filtros opcionales y parámetros repetidos. `MultiValueMap` o `List` capturan múltiples valores. Deben documentarse valores por defecto, rango y semántica de ausencia.

En el snapshot acumulativo de M3 esta idea no queda como teoría aislada: se refleja en el controlador, el servicio, los DTO y los tests que corresponden a este punto. La comprobación debe hacerse sobre el comportamiento HTTP observable y no sólo leyendo anotaciones.

Pregunta

¿Qué decisión de contrato de **@RequestParam en profundidad** debería poder demostrar un test o una petición reproducible?

Respuesta razonada

Los query parameters permiten filtros opcionales y parámetros repetidos. `MultiValueMap` o `List` capturan múltiples valores. Deben documentarse valores por defecto, rango y semántica de ausencia. La evidencia correcta combina una entrada concreta con el status, las cabeceras y/o el cuerpo esperados, y deja claro qué capa aplica la regla.

Bloque 2 - @RequestHeader

T2.1 - Qué son las cabeceras HTTP

Las cabeceras HTTP son pares clave-valor que acompañan a una petición o a una respuesta. Llevan metadatos: quién envía la petición, qué formato acepta, qué credenciales usa, etc. En una petición, las cabeceras más comunes son:

- Host: el dominio del servidor.
- User-Agent: el cliente que envía la petición.
- Accept: el formato de respuesta que el cliente acepta.
- Content-Type: el formato del cuerpo que el cliente envía.
- Authorization: credenciales (por ejemplo, un token).
- Cookie: cookies. En una respuesta, las cabeceras más comunes son:
- Content-Type: el formato del cuerpo devuelto.
- Content-Length: el tamaño del cuerpo.
- Location: la URL del recurso creado (en POST).
- Cache-Control: directivas de caché.
- Set-Cookie: cookies que el cliente debe guardar.

T2.2 - @RequestHeader

`@RequestHeader` extrae una cabecera de la petición y la inyecta como parámetro del método:

```
@GetMapping
public List<AlumnoDTO> listar(
    @RequestHeader(value = "User-Agent", required = false) String userAgent) {
    // ...
}
```

Si el cliente envía la cabecera User-Agent, el parámetro la contendrá. Si no, será null (porque required = false). Al igual que @RequestParam, @RequestHeader puede capturar todas las cabeceras en un mapa:

```
@GetMapping
public List<AlumnoDTO> listar(
    @RequestHeader Map<String, String> cabeceras) {
    // ...
}
```

El mapa contendrá todas las cabeceras de la petición: {"host": "localhost:8080", "user-agent": "...", ...}.

Pregunta: ¿Para qué sirve leer el User-Agent en un controlador? ¿Qué información aporta?

T2.3 - Casos de uso típicos

Las cabeceras se usan para varias cosas: Autenticación. La cabecera Authorization lleva el token de autenticación. En el Módulo 6, cuando implementemos JWT, la leeremos para validar el token. Negociación de contenido. La cabecera Accept indica qué formato acepta el cliente. Spring MVC la usa para decidir si devolver JSON o XML. Idioma. La cabecera Accept-Language indica el idioma preferido del cliente. Se usa para devolver mensajes en el idioma correcto. Trazabilidad. Cabeceras como X-Request-Id o X-Correlation-Id permiten seguir una petición a través de varios servicios. Idempotencia. La cabecera Idempotency-Key permite al cliente indicar que una petición POST es un reintento, y evitar que se cree un recurso duplicado. En este curso, las cabeceras más importantes son Authorization (para JWT) y Content-Type (para indicar el formato del cuerpo).

Pregunta de cierre del bloque: ¿Qué cabecera usarías para indicar que una petición es un reintento y evitar duplicados?

Actualización 2026 y vínculo con el proyecto

Las cabeceras transportan metadatos de la petición. User-Agent y Accept-Language sirven para observar el protocolo; cabeceras propias deben tener nombres estables y no sustituir datos de dominio del cuerpo.

En el snapshot acumulativo de M3 esta idea no queda como teoría aislada: se refleja en el controlador, el servicio, los DTO y los tests que corresponden a este punto. La comprobación debe hacerse sobre el comportamiento HTTP observable y no sólo leyendo anotaciones.

Pregunta

¿Qué decisión de contrato de @requestheader debería poder demostrar un test o una petición reproducible?

Respuesta razonada

Las cabeceras transportan metadatos de la petición. User-Agent y Accept-Language sirven para observar el protocolo; cabeceras propias deben tener nombres estables y no sustituir datos de dominio del cuerpo. La evidencia correcta combina una entrada concreta con el status, las cabeceras y/o el cuerpo esperados, y deja claro qué capa aplica la regla.

Bloque 3 - @CookieValue y @MatrixVariable

T3.1 - Qué son las cookies

Las cookies son pequeños fragmentos de información que el servidor envía al cliente y que el cliente devuelve en cada petición posterior. Se usan para mantener el estado entre peticiones (aunque HTTP es sin estado). Una cookie tiene:

- Nombre y valor. Por ejemplo, sessionId=abc123.
- Dominio y ruta. A qué dominio y ruta aplica.
- Fecha de expiración. Cuándo deja de ser válida.
- Flags de seguridad. Secure (solo HTTPS), HttpOnly (no accesible desde JavaScript), SameSite (controla envío entre sitios). En APIs REST, las cookies se usan menos que en aplicaciones web tradicionales. La autenticación suele hacerse con tokens en la cabecera Authorization, no con cookies. Pero conviene saber cómo leerlas.

T3.2 - @CookieValue

@CookieValue extrae una cookie de la petición y la inyecta como parámetro:

```
@GetMapping
public List<AlumnoDTO> listar(
    @CookieValue(value = "sessionId", required = false) String sessionId) {
    // ...
}
```

Si el cliente envía la cookie sessionId, el parámetro la contendrá. Si no, será null (porque required = false). Al igual que las otras anotaciones, @CookieValue puede capturar todas las cookies en un mapa:

```
@GetMapping
public List<AlumnoDTO> listar(
    @CookieValue Map<String, String> cookies) {
    // ...
}
```

Pregunta: ¿Por qué en APIs REST se usan menos las cookies que en aplicaciones web tradicionales?

T3.3 - @MatrixVariable

@MatrixVariable es una anotación menos común que permite extraer variables de la ruta con un formato especial. Por ejemplo:

GET /api/v1/alumnos;curso=5º;turno=mañana

Los parámetros separados por ; se llaman matrix variables. Se capturan con @MatrixVariable:

```
@GetMapping("/alumnos/{filtros}")
public List<AlumnoDTO> listar(
    @MatrixVariable(pathVar = "filtros") Map<String, String> filtros) {
    // ...
}
```

Las matrix variables son poco comunes en APIs REST. Se usan cuando se quiere combinar filtros en la ruta sin usar query parameters. En la práctica, la mayoría de las APIs usan query parameters. Pero conviene conocer que existen.

Pregunta de cierre del bloque: ¿Qué diferencia hay entre ?curso=5º y ;curso=5º? ¿Cuándo usarías cada uno?

Actualización 2026 y vínculo con el proyecto

CookieValue extrae cookies concretas. Matrix variables existen pero son poco habituales en APIs REST modernas; los query parameters suelen ser más interoperables y previsibles.

En el snapshot acumulativo de M3 esta idea no queda como teoría aislada: se refleja en el controlador, el servicio, los DTO y los tests que corresponden a este punto. La comprobación debe hacerse sobre el comportamiento HTTP observable y no sólo leyendo anotaciones.

Pregunta

¿Qué decisión de contrato de **@cookievalue** y **@matrixvariable** debería poder demostrar un test o una petición reproducible?

Respuesta razonada

CookieValue extrae cookies concretas. Matrix variables existen pero son poco habituales en APIs REST modernas; los query parameters suelen ser más interoperables y previsibles. La evidencia correcta combina una entrada concreta con el status, las cabeceras y/o el cuerpo esperados, y deja claro qué capa aplica la regla.

Bloque 4 - Postman

T4.1 - Qué es Postman y por qué usarlo

Postman es una herramienta gráfica para probar APIs. Permite enviar peticiones HTTP de cualquier tipo (GET, POST, PUT, DELETE...), con cualquier cabecera y cualquier cuerpo, y ver la respuesta de forma clara. Postman tiene varias ventajas frente a curl:

- Interfaz gráfica. No hay que recordar flags ni sintaxis.
- Organización. Se pueden guardar peticiones en colecciones.
- Entornos. Se pueden definir variables de entorno (por ejemplo, baseUrl) y cambiar entre desarrollo, test y producción.
- Tests automáticos. Se pueden escribir scripts que verifiquen la respuesta.
- Historial. Se guardan todas las peticiones enviadas.
- Compartición. Las colecciones se pueden exportar e importar. Postman se descarga desde postman.com/downloads/. Hay versión para Windows, Linux y macOS. La versión gratuita es suficiente para este curso.

T4.2 - Colecciones y entornos

Una colección es un grupo de peticiones organizadas. Por ejemplo, se puede crear una colección “Mi Proyecto” con subcarpetas para “Alumnos”, “Expedientes”, etc. Cada petición tiene:

- Método HTTP.
- URL.
- Cabeceras.
- Cuerpo.
- Scripts de test. Un entorno es un conjunto de variables que se pueden usar en las peticiones. Por ejemplo, se define una variable baseUrl con valor `http://localhost:8080` y se usa `{{baseUrl}}/api/v1/alumnos` en las URLs. Si mañana cambia el puerto, solo hay que cambiar la variable. Los entornos permiten tener varios conjuntos de variables: dev, test, prod. Se cambia de entorno con un desplegable. **Pregunta: ¿Qué ventaja tiene usar variables de entorno en lugar de escribir la URL completa en cada petición?**

T4.3 - Tests automáticos en Postman

Postman permite escribir scripts de test que se ejecutan después de cada petición. Se escriben en JavaScript y verifican la respuesta:

```

pm.test("Status code is 200", function () {
    pm.response.to.have.status(200);
});

pm.test("Response has id", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property("id");
});

pm.test("Nombre es Ana", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData.nombre).to.eql("Ana");
});

```

Los tests verifican:

- pm.response.to.have.status(200) que el código es 200.
- pm.expect(jsonData).to.have.property("id") que la respuesta tiene el campo id.
- pm.expect(jsonData.nombre).to.eql("Ana") que el nombre es "Ana". Si un test falla, Postman lo marca en rojo. Es una forma cómoda de verificar que la API responde correctamente. **Pregunta de cierre del bloque: ¿Qué ventaja tiene escribir tests en Postman en lugar de probar manualmente?**

Actualización 2026 y vínculo con el proyecto

Una colección agrupa peticiones; un entorno aporta baseUrl y variables. Los scripts de test permiten comprobar status, headers y JSON, complementando curl y los tests Java.

En el snapshot acumulativo de M3 esta idea no queda como teoría aislada: se refleja en el controlador, el servicio, los DTO y los tests que corresponden a este punto. La comprobación debe hacerse sobre el comportamiento HTTP observable y no sólo leyendo anotaciones.

Pregunta

¿Qué decisión de contrato de **postman** debería poder demostrar un test o una petición reproducible?

Respuesta razonada

Una colección agrupa peticiones; un entorno aporta baseUrl y variables. Los scripts de test permiten comprobar status, headers y JSON, complementando curl y los tests Java. La evidencia correcta combina una entrada concreta con el status, las cabeceras y/o el cuerpo esperados, y deja claro qué capa aplica la regla.

Bloque 5 - Buenas prácticas con parámetros y cabeceras

T5.1 - Buenas prácticas con @RequestParam

Usar defaultValue para parámetros opcionales. Evita tener que comprobar null en el código. Validar los tipos. Si un parámetro es numérico, declararlo como int. Spring convierte y devuelve 400 si falla. Documentar los parámetros. En OpenAPI, describir qué parámetros acepta cada endpoint. No usar demasiados parámetros. Si un endpoint tiene 10 parámetros, probablemente está haciendo demasiado. Usar listas para parámetros múltiples. ?curso=5º&curso=6º en lugar de ?curso=5º,6º.

T5.2 - Buenas prácticas con @RequestHeader

No leer cabeceras innecesarias. Solo las que se usan. Usar required = false para cabeceras opcionales. No todas las cabeceras están siempre presentes. Documentar las cabeceras personal-

izadas. Si la API usa X-Mi-Cabecera, documentarla. No confiar en cabeceras para seguridad. Las cabeceras se pueden falsificar. Para seguridad, usar tokens firmados. Leer Authorization para autenticación. Es la cabecera estándar.

T5.3 - Buenas prácticas con Postman

Organizar las peticiones en colecciones. Agrupar por recurso o por funcionalidad. Usar variables de entorno. Facilitan cambiar entre entornos. Escribir tests. Verifican que la API responde correctamente. Guardar las colecciones en el repositorio. Para que el equipo las comparta. No guardar credenciales en las peticiones. Usar variables de entorno y no compartir las que tengan secretos. Exportar e importar colecciones. Para compartir con otros desarrolladores.

Pregunta de cierre del bloque: ¿Por qué no se deben guardar credenciales directamente en las peticiones de Postman?

Actualización 2026 y vínculo con el proyecto

La observación completa combina método, URL, query, headers, cookies, status y cuerpo. Herramientas gráficas no sustituyen un contrato automatizado; sirven para explorar, reproducir y comunicar peticiones.

En el snapshot acumulativo de M3 esta idea no queda como teoría aislada: se refleja en el controlador, el servicio, los DTO y los tests que corresponden a este punto. La comprobación debe hacerse sobre el comportamiento HTTP observable y no sólo leyendo anotaciones.

Pregunta

¿Qué decisión de contrato de **buenas prácticas con parámetros y cabeceras** debería poder demostrar un test o una petición reproducible?

Respuesta razonada

La observación completa combina método, URL, query, headers, cookies, status y cuerpo. Herramientas gráficas no sustituyen un contrato automatizado; sirven para explorar, reproducir y comunicar peticiones. La evidencia correcta combina una entrada concreta con el status, las cabeceras y/o el cuerpo esperados, y deja claro qué capa aplica la regla.

Resumen del punto

- @RequestParam: extrae query parameters. Opcional con required = false o defaultValue.
- Listas y mapas: @RequestParam List y @RequestParam Map<String, String>.
- @RequestHeader: extrae cabeceras. Opcional con required = false.
- @CookieValue: extrae cookies.
- @MatrixVariable: extrae variables de la ruta con ;.
- Cabeceras comunes: Host, User-Agent, Accept, Content-Type, Authorization, Cookie.
- Postman: herramienta gráfica para probar APIs.
- Colecciones: agrupan peticiones.
- Entornos: variables para cambiar entre dev/test/prod.
- Tests: scripts en JavaScript que verifican la respuesta.

Puente a la práctica

En la práctica 3.6 estas ideas se aplican sobre el mismo proyecto heredado de M2. Antes de continuar al siguiente punto se conserva la regresión anterior y se añade evidencia automatizada para el nuevo contrato.